

# Documentation technique de référence

DOCUMENTATION TECHNIQUE

## Plateforme HEdS

Documentation technique de reprise pour comprendre l'architecture, remettre l'environnement en route, maintenir le code et faire évoluer la plateforme.

Démarrage

Architecture

Carte du repo

Vue 3

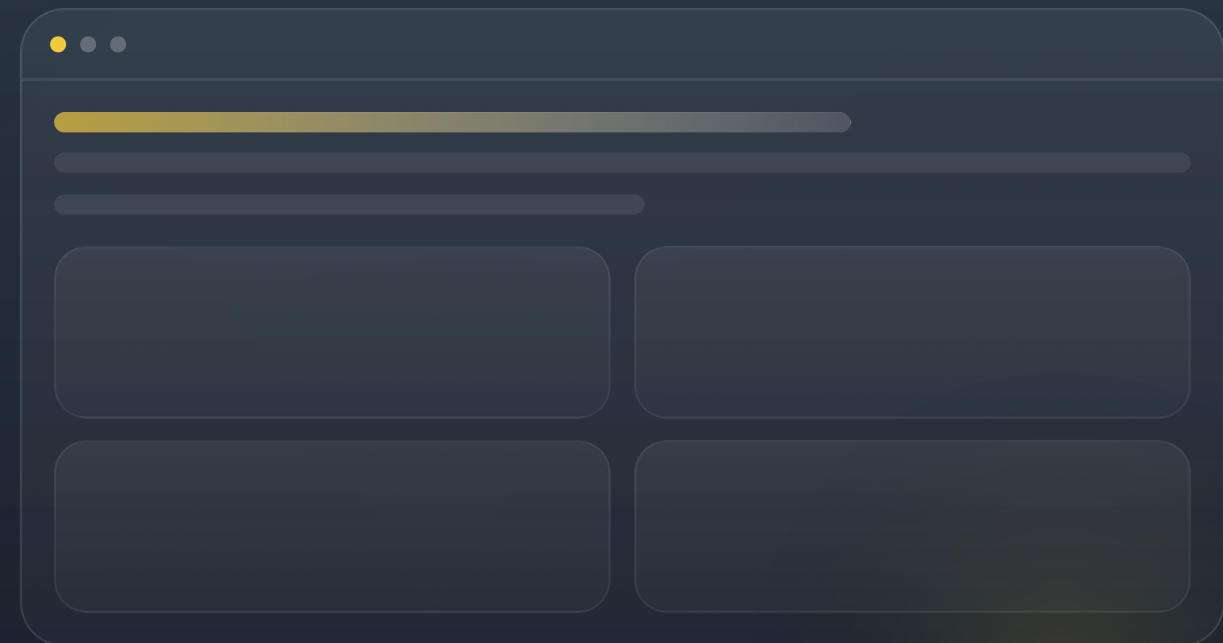
Vite

PrimeVue

Firebase

Supabase

Express



◆ Architecture réelle

◆ Flux métier

◆ Sources de données

◆ Exploitation & runbook

## Finalité

Cette documentation doit permettre à une nouvelle personne technique de :

- relancer le projet

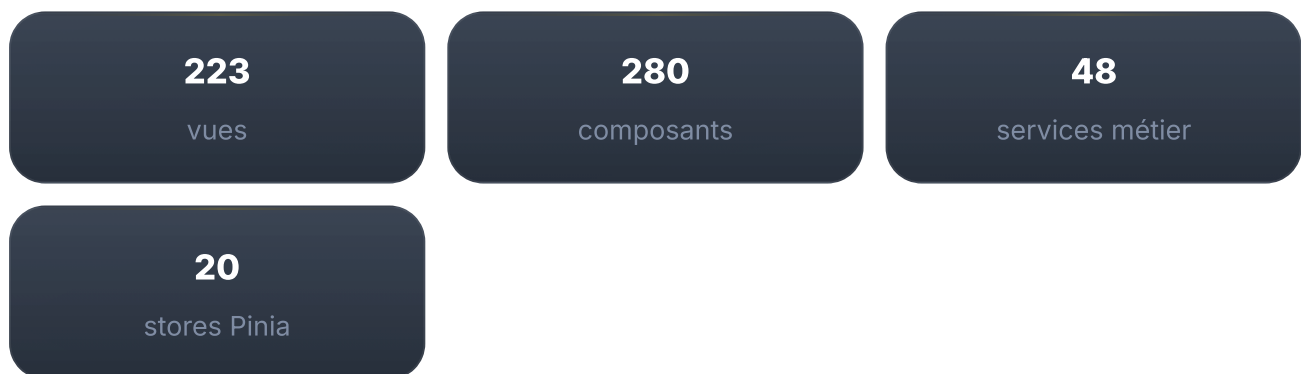
- comprendre l'architecture
- retrouver rapidement où modifier une fonctionnalité
- déployer et diagnostiquer la plateforme

## Réalité du projet

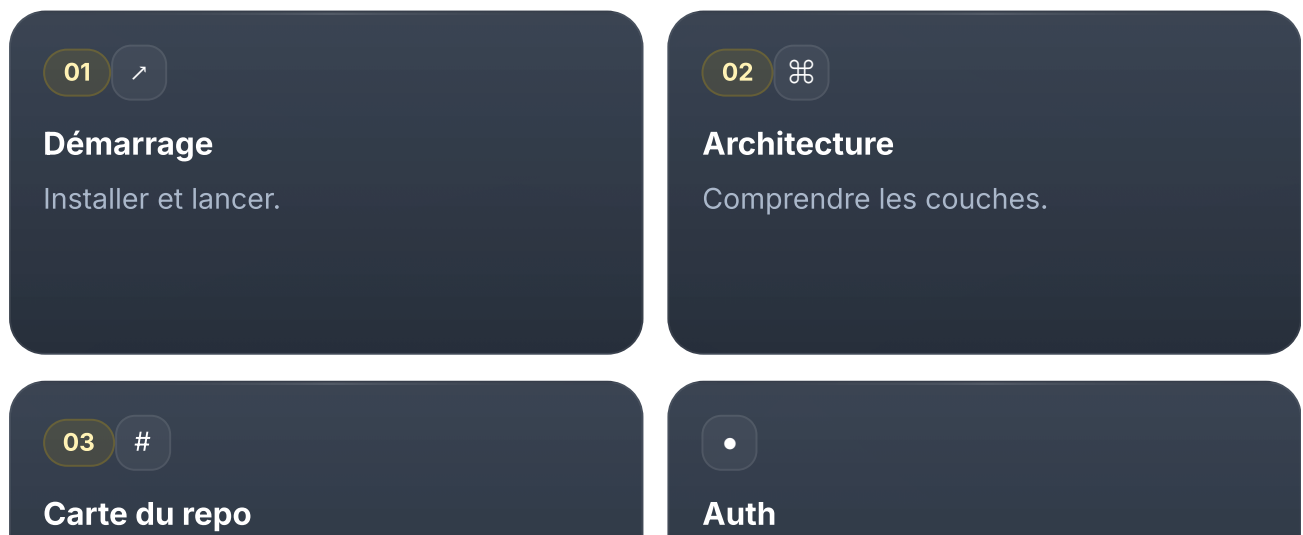
La plateforme regroupe plusieurs sous-systèmes dans une seule base de code :

- frontend Vue 3
- administration académique
- formation pratique
- workflows soins infirmiers
- social et gamification
- applications intégrées
- backend Node
- coexistence Firebase et Supabase

## Chiffres utiles



## Lecture recommandée



Retrouver le bon code.

Session, rôles, permissions.



## Données

Firebase, Supabase, migration.



## Déploiement

Build app et docs.

# Démarrage rapide et reprise du projet

## POINT D'ENTRÉE

Cette page sert de base de reprise pour relancer le projet avant d'entrer dans les détails techniques.

01



### Installer

Récupérer les dépendances du projet et de la documentation.

02



### Configurer

Préparer les variables d'environnement et les accès externes.

03



### Lancer

Démarrer l'application, puis la documentation locale.

## Étapes minimales

1. installer les dépendances
2. configurer l'environnement
3. lancer l'application
4. lancer la documentation

## Commandes utiles

```
npm install
npm run dev
npm run docs:dev
```



## Lire ensuite

- [architecture](#)
- [system/project-structure](#)
- [ops/development](#)
- [auth/overview](#)

# Architecture globale

## VUE D'ENSEMBLE

Toute reprise du projet doit relier l'interface, la logique métier et la vraie source de données.

- ◆ Frontend Vue 3
- ◆ Services métier & stores
- ◆ Firebase, Supabase, backend

## Résumé

Le projet est une application Vue 3 qui agrège plusieurs domaines métier et utilise :

- Firebase pour une partie legacy
- Supabase comme cible principale
- un backend Node/Express pour certaines API et intégrations

## Schéma

## Fichiers structurants

- `src/main.js`
- `src/router.js`
- `src/firebase.js`
- `src/supabase.js`
- `backend/index.js`
- `supabase/migrations/`

# Vue système et domaines fonctionnels

## Vue d'ensemble

La plateforme regroupe plusieurs domaines fonctionnels dans une seule base de code.

## Domaines principaux

- administration et académique
- formation pratique
- planning et soins
- social et gamification
- applications intégrées

## Point de reprise

Quand on reprend le projet, il faut toujours raisonner en trois couches :

1. interface et navigation
2. services et logique métier
3. source de données réelle

# Checklist de reprise technique

## REPRISE PROJET

Cette checklist sert à reprendre la plateforme de façon méthodique, sans dépendre uniquement d'explications orales.

### 1. Vérifier que le projet démarre

- installer les dépendances racine ;
- démarrer l'application avec `npm run dev` ;
- démarrer la documentation avec `npm run docs:dev` ;
- confirmer que l'app répond et que la doc s'ouvre correctement.

### 2. Vérifier les variables d'environnement

Confirmer la présence et la cohérence de :

- `VITE_SUPABASE_URL`
- `VITE_SUPABASE_KEY`
- `VITE_FIREBASE_*`
- `VITE_DISABLE_AUTH`
- `VITE_DEBUG_ROUTER`

Contrôler aussi si le backend possède son propre `.env`.

### 3. Identifier le mode d'auth réel

Le projet peut fonctionner selon plusieurs cas :

- Supabase comme provider principal ;
- Firebase encore actif pour certains flux legacy ;
- bypass auth via `VITE_DISABLE_AUTH=true`.

À valider dans :

- `src/stores/authStore.js`
- `src/stores/role.js`
- `src/router.js`

## 4. Valider le cycle permissions

Points à vérifier :

- la session Supabase existe bien ;
- `roleStore.init()` est appelé ;
- la RPC `api_my_permissions` répond ;
- le fallback `user_profiles.role` / `user_profiles.permissions` fonctionne ;
- `super.all` est bien pris en compte si utilisé.

## 5. Repérer les couches actives

Le projet se lit par couches :

1. vues dans `src/views/`
2. stores dans `src/stores/`
3. services dans `src/service/`
4. backend `backend/` si nécessaire
5. source de données réelle Firebase ou Supabase

## 6. Cartographier les zones métier critiques

Priorité de lecture conseillée :

- formation pratique
- planning et soins infirmiers
- auth / permissions
- backend / routes API
- données et migrations Supabase

## 7. Vérifier les routes sensibles

Routes à auditer en premier :

- pages protégées par `meta.need`
- routes historiques avec `requiredRole`

- routes dynamiques injectées depuis Supabase
- redirections `/`, `/home`, `/feed`, `/access`

Fichiers clés :

- `src/router.js`
- `src/router/routes/index.js`
- `src/router/routes/*.js`

## 8. Vérifier le backend actif

Confirmer :

- que `backend/index.js` est la vraie entrée serveur ;
- que les endpoints critiques répondent ;
- que les routes listées dans la doc correspondent encore au code ;
- que les scripts imports / seeds ne sont pas confondus avec les routes actives.

## 9. Vérifier les données et migrations

À faire avant toute modification métier :

- lire `supabase/migrations/`
- repérer les migrations récentes touchant le domaine
- identifier si le domaine est encore partiellement en Firebase
- valider les politiques RLS

## 10. Vérifier les outils de diagnostic

- `/admin/supabase-diagnostic`
- diagnostic Firebase dans l'admin
- `/health`
- `/api/ping`
- `/api/ftp/diagnostic`

## 11. Vérifier le build et la livraison

Commandes à exécuter :



```
npm run docs:build
npm run build
npm run build:all
```

Contrôler :

- génération de `dist/`
- génération de `documentation/build/`
- copie de la doc vers `dist/docs`

## 12. Vérifier le niveau de dette

Signaux de dette à noter immédiatement :

- mélange `need` / `requiredRole`
- coexistence Firebase / Supabase sur un même flux
- scripts de préparation de données hors UI
- copies legacy dans `backend/`
- différences entre comportement réel et documentation

## Sortie attendue

Une reprise correcte doit produire au minimum :

- une app qui démarre ;
- une doc qui build ;
- une cartographie claire des zones critiques ;
- une compréhension du couple auth + routing + permissions ;
- une liste des domaines encore legacy ou sensibles.

# Carte du repository

## Vue d'ensemble

```
pfpheds/  
├── src/  
├── backend/  
├── documentation/  
├── supabase/  
├── scripts/  
├── tests/  
├── public/  
└── docs/
```

## Frontend

- `src/router.js`
- `src/router/routes/`
- `src/stores/`
- `src/service/`
- `src/views/`
- `src/components/`

## Backend

- `backend/index.js`
- `backend/supabase/`
- `backend/uploads/`
- `backend/workers/`



# Bootstrap applicatif

## Point d'entree

Le bootstrap se fait dans `src/main.js`.

## Responsabilites principales

1. creation de l'application
2. installation de Pinia et du router
3. services PrimeVue
4. gestion service worker
5. initialisation auth et user store

# Bibliothèque de composants - vue d'ensemble

## COMPONENT LIBRARY

Cette page sert de point d'entrée pour naviguer dans la bibliothèque de composants réelle du projet.

## Familles observées dans `src/components/`

Répertoires principaux :

- `academic`
- `admin`
- `capsulesInsuffiance`
- `careconvers`
- `common`
- `editor`
- `events`
- `games`
- `gamification`
- `home`
- `media`
- `plugins`
- `Profile`
- `push`
- `social`
- `tournois`
- `ui`
- `user`
- `video`

## Grandes catégories utiles

Pour reprendre le front, il faut distinguer :

- composants de layout global ;
- composants utilitaires communs ;
- composants admin réutilisables ;
- composants métiers lourds ;
- composants social/media ;
- composants de profil et gamification.

## Ce qui ressemble à une vraie bibliothèque réutilisable

Les familles les plus "bibliothèque" sont :

- `src/layout/`
- `src/components/common/`
- `src/components/ui/`
- `src/components/admin/widgets/`
- `src/components/events/`
- `src/components/video/`

## Ce qui ressemble plus à des composants métier

- `src/components/admin/details/`
- `src/components/admin/forms/`
- `src/components/social/library/`
- `src/components/gamification/`
- `src/components/user/details/`

## Lecture recommandée

1. `layout` pour comprendre la coque globale ;
2. `common` et `ui` pour les briques génériques ;
3. `admin/widgets` pour les dashboards et KPI ;
4. `social/library` pour le sous-système social ;
5. `events`, `video`, `gamification` pour les familles spécialisées.

## Signaux de dette visibles

Le dossier composants montre aussi quelques reliquats ou artefacts :

- fichiers de test ou temporaires ;
- composants historiques aux noms hétérogènes ;
- doublons de logique entre composants métier et vues.

Cela ne veut pas dire qu'ils sont supprimables sans audit, mais il faut les repérer comme dette potentielle.

# Bibliothèque de composants - layout global

## Répertoire principal

- `src/layout/`

## Composants observés

- `AppLayout.vue`
- `AppSidebar.vue`
- `AppTopbar.vue`
- `AppMenu.vue`
- `AppMenuItem.vue`
- `AppSubMenu.vue`
- `AppBreadcrumb.vue`
- `AppProfileSidebar.vue`
- `AppConfig.vue`
- `AppDarkAndLightMode.vue`

## Rôle global

Ces composants portent la structure de navigation principale :

- shell de l'application ;
- sidebar ;
- topbar ;
- menus ;
- configuration visuelle ;
- navigation secondaire.

## Composants structurants

### `AppLayout.vue`

Point d'ancrage du layout principal. À auditer en premier quand un problème touche :

- structure générale ;
- overlays ;

- comportement responsive ;
- injection de slot/page.

### **AppSidebar.vue**

Responsable de la navigation latérale, directement liée au menu applicatif.

### **AppTopbar.vue**

Responsable des actions hautes, accès utilisateur, recherche ou raccourcis selon la configuration courante.

### **AppMenu.vue / AppMenuItem.vue / AppSubMenu.vue**

Portent la logique de rendu du menu et de ses sous-niveaux.

## **Réflexes de maintenance**

Quand un bug touche la navigation :

1. vérifier la route active ;
2. vérifier le layout ;
3. vérifier la sidebar ;
4. vérifier les composants de menu ;
5. vérifier ensuite les permissions ou le masquage de certaines entrées.

# Bibliothèque de composants - communs et UI

## Répertoires concernés

- `src/components/common/`
- `src/components/ui/`

## Familles communes observées

### Feedback / états

- `AppSkeleton.vue`
- `AppSpinner.vue`
- `EmptyState.vue`
- `ErrorInline.vue`

Usage :

- chargement ;
- absence de données ;
- erreur locale ;
- feedback visuel standard.

### Filtres

- `FilterSidebar.vue`
- `FilterInstitution.vue`
- `FiltreMap.vue`

Usage :

- filtrage transverse ;
- filtres institution/cartographie ;
- navigation contextuelle.

### Formulaires

- `AuthForm.vue`

Usage :

- auth et formulaires simples structurés.

## Utilitaires UI

- `Error404.vue`
- `Footer.vue`
- `GlobalSearch.vue`
- `HeaderIcons.vue`
- `Loader.vue`
- `MobileBottomNav.vue`
- `Navbar.vue`
- `PwaInstallPrompt.vue`
- `SearchResults.vue`
- `VersionningComponent.vue`

## UI générique

- `BlockViewer.vue`
- `QrCodeGenerator.vue`
- `ButtonNavbar.vue`
- `SearchButton.vue`
- `SwitchColor.vue`

## Réflexes de reprise

Quand un composant est purement réutilisable, documenter :

- props ;
- emits ;
- dépendances à PrimeVue / styles ;
- dépendances à des stores si elles existent ;
- usages principaux dans les vues.

## Piège courant

Des composants apparemment "UI" peuvent déjà embarquer une logique métier légère. Vérifier avant de les traiter comme purement présentatifs.



# Bibliothèque de composants - admin et widgets

## Répertoires concernés

- `src/components/admin/common/`
- `src/components/admin/layouts/`
- `src/components/admin/widgets/`
- `src/components/admin/places/`
- `src/components/admin/forms/`
- `src/components/admin/details/`

## Blocs réutilisables

### Headers

- `AdminPageHeader.vue`
- `PageHeader.vue`

Rôle :

- unifier les titres ;
- exposer les actions principales ;
- éviter que chaque vue admin recrée sa propre structure d'en-tête.

### Layout admin

- `AdminLayout.vue`
- `AdminSidebar.vue`
- `SidebarMenuItems.vue`

Rôle :

- fournir une coque commune ;
- encapsuler la navigation et les zones d'actions ;
- stabiliser le rendu entre les sections admin.

### Widgets KPI / dashboard

- `StatsCard.vue`

- KpiCard.vue
- AlertsWidget.vue
- DashboardKpiGrid.vue
- MiniChart.vue
- PeriodSelector.vue
- PeriodComparisonPanel.vue
- PfpCohortKpiWidget.vue
- PfpComparisonView.vue
- SmartVisualization.vue
- ResizableWidgetGrid.vue

Rôle :

- construire les dashboards académiques, PFP et analytiques ;
- encapsuler l'affichage des indicateurs ;
- centraliser la logique de comparaison, de personnalisation et d'export visuel.

## Visualisations élémentaires

- widgets/charts/BarChart.vue
- widgets/charts/LineChart.vue
- widgets/charts/PieChart.vue
- widgets/charts/DoughnutChart.vue
- widgets/charts/SimpleTable.vue

Rôle :

- fournir des briques de base branchées sur les widgets plus riches ;
- éviter de dupliquer la configuration des graphes.

## Dialogs et formulaires

- CreatePlaceDialog.vue
- StudentCreateDialog.vue
- StudentEditDialog.vue
- EnseignantForm\*.vue
- EtudiantForm\*.vue
- InstitutionForm\*.vue
- NewUserForm\*.vue

Rôle :

- piloter les flux CRUD les plus fréquents ;
- concentrer validation, mapping et émission des événements.

## Couplage réel

Ces composants sont rarement purement visuels. Ils portent souvent :

- des hypothèses de schéma ;
- des noms de champs historiques ;
- des branches métier liées aux rôles ;
- des adaptations PFP spécifiques.

En pratique, un widget admin doit être traité comme un composant métier, pas comme une simple brique d'interface.

## Composants à lire en priorité

`src/components/admin/widgets/DashboardKpiGrid.vue`

Ce composant gère :

- la personnalisation locale du dashboard ;
- le drag and drop HTML5 ;
- la visibilité des KPI ;
- l'export / import JSON ;
- le stockage local de la configuration ;
- le mode comparaison.

Conséquence : si un dashboard semble purement visuel, le bug peut en réalité venir d'un état local persistant, pas seulement des données.

`src/components/admin/widgets/KpiCard.vue`

Carte KPI pivot. C'est la couche d'affichage qui reçoit la plupart des props enrichies.

`src/components/admin/places/CreatePlaceDialog.vue`

Composant critique pour la création de places. Il est couplé :

- aux stores de places ;
- aux institutions ;
- aux praticiens formateurs ;
- au mapping métier des champs PFP.

**src/components/admin/details/\***

Sous-dossier hétérogène, très métier, à auditer avec prudence. Plusieurs composants y embarquent encore des conventions historiques et des noms de champs peu homogènes.

## Réflexe de maintenance

Quand une page admin casse :

1. vérifier la vue ;
2. vérifier le composant métier branché dans la vue ;
3. vérifier le store ou service qui l'alimente ;
4. vérifier le contrat de données attendu ;
5. vérifier si le problème vient d'un reliquat legacy dans les noms de champs.

# Bibliothèque de composants - social, médias, événements

## Répertoires concernés

- `src/components/social/library/`
- `src/components/events/`
- `src/components/video/`
- `src/components/media/`

## Social library

Composants notables :

- `MainFeed.vue`
- `MainFeedSupabase.vue`
- `PostItem.vue`
- `CreatePostDialog.vue`
- `PostTextarea.vue`
- `CommunitiesList.vue`
- `CommunityFeed.vue`
- `StoriesBar.vue`
- `StoryModal.vue`
- `StoryEditor.vue`
- `AddStoryCore.vue`
- `InfinityScroll.vue`
- `ImageCropper.vue`
- `VotationBanner.vue`

## Ce que porte cette famille

- feed ;
- publication de contenu ;
- stories ;
- communautés ;
- édition média ;
- scroll infini ;

- UI sociale mobile.

## Points d'attention

- cette famille est dense et très couplée au social backend ;
- certains composants mélangent UI, upload, édition et logique de publication ;
- `MainFeedSupabase.vue` et les composants associés méritent une lecture attentive en reprise.

## Événements

Composants notables :

- `EventCard.vue`
- `EventDetail.vue`
- `EventForm.vue`

Usage :

- affichage synthétique ;
- détail ;
- création/édition d'événements.

## Vidéo / média

Composants notables :

- `VideoCard.vue`
- `ModuleSection.vue`
- `VideoPlayerVimeo.vue`

Usage :

- bibliothèque vidéo ;
- modules média ;
- lecture Vimeo.

## Réflexe de reprise

Quand un bug touche ces composants :

1. identifier si le problème est UI, upload, API ou storage ;
2. vérifier les services associés ;
3. vérifier si le composant est utilisé en web, mobile ou admin ;
4. vérifier les dépendances Vimeo / médias / backend social.

# Bibliothèque de composants - composants critiques

## LECTURE PRIORITAIRE

Cette page liste les composants qui concentrent le plus de logique, de réutilisation ou de couplage métier.

### Priorité 1 — structure globale

`src/layout/AppLayout.vue`

À lire si :

- le shell global casse ;
- le responsive global est instable ;
- le comportement du layout semble incohérent.

Responsable de :

- l'assemblage `AppSidebar` / `AppTopbar` / `router-view` ;
- les classes de layout calculées depuis `useLayout()` ;
- la fermeture hors clic du menu overlay.

Point de vigilance :

- la fonction `unbindOutsideClickListener` retire le listener avec une référence potentiellement incorrecte ; c'est un point à vérifier si des comportements fantômes apparaissent.

`src/layout/AppSidebar.vue`

À lire si :

- la navigation latérale pose problème ;
- des entrées disparaissent ;



- l’affichage menu / permissions devient incohérent.

**src/layout/AppTopbar.vue**

À lire si :

- les actions hautes, raccourcis ou accès utilisateur posent problème.

## Priorité 2 — auth / navigation visible

**src/components/common/forms/AuthForm.vue**

Composant clé pour le tunnel d’entrée utilisateur.

Contrat principal :

- props : email, password, remember, loading, états d’erreur ;
- events : update:email, update:password, update:remember, submit, reset-password.

Lecture utile si :

- la page de login affiche mal les erreurs ;
- le binding du formulaire ne remonte plus ;
- un refactoring auth a cassé les événements attendus par la vue parente.

**src/components/common/utils/MobileBottomNav.vue**

Important pour la navigation mobile.

**src/components/common/utils/GlobalSearch.vue**

À surveiller si la recherche transversale casse.

## Priorité 3 — admin réutilisable

**src/components/admin/layouts/AdminLayout.vue**

Coque admin réutilisable.

**src/components/admin/common/AdminPageHeader.vue**

Standardise les pages admin.

**src/components/admin/widgets/DashboardKpiGrid.vue**

Composant structurant pour plusieurs dashboards.

Responsabilités notables :

- ordre des cartes ;
- masquage ;
- tailles ;
- comparaison ;
- export / import ;
- persistance locale.

**src/components/admin/widgets/StatsCard.vue**

Carte KPI simple mais très centrale.

**src/components/admin/widgets/SmartVisualization.vue**

Composant potentiellement dense, avec logique de visualisation et d'export.

**src/components/admin/places/CreatePlaceDialog.vue**

Critique si le flux de création de places change.

## Priorité 4 — social

**src/components/social/library/MainFeedSupabase.vue**

Probablement un des composants sociaux les plus critiques.

Responsabilités notables :

- chargement initial des posts ;
- filtres hashtag / communauté ;
- upload média ;
- abonnement temps réel ou polling ;
- intégration gamification ;
- mode mobile / desktop.

**src/components/social/library/PostItem.vue**

Brique cœur du feed.

`src/components/social/library/CreatePostDialog.vue`

À lire si la publication casse.

`src/components/social/library/CommunityFeed.vue`

À lire si le flux communauté casse.

`src/components/social/library/StoryModal.vue`

À lire si les stories posent problème.

## Priorité 5 — événements / médias

`src/components/events/EventForm.vue`

Critique pour la création / édition d'événements.

Contrat principal :

- props : `event`, `editMode`, `userId` ;
- events : `submit`, `close`, `delete`.

Point sensible :

- le composant gère à la fois création, modification et suppression ;
- il mélange validation, preview image et contrôle d'autorisation.

Dette ou bug probable :

- la suppression émet `event?.id` alors que `event` n'est pas déclaré dans le scope script ; il faut probablement utiliser `props.event?.id`.

`src/components/events/EventDetail.vue`

Dense, utile pour les interactions événementielles.

`src/components/video/VideoCard.vue`

Brique de base de la bibliothèque vidéo.

`src/components/media/VideoPlayerVimeo.vue`

Critique si la lecture Vimeo casse.

## Priorité 6 — profil / gamification

`src/components/gamification/QuestCard.vue`

Carte métier centrale côté quêtes.

`src/components/gamification/ChallengeCard.vue`

Carte critique côté défis.

`src/components/gamification/BandeauMaison.vue`

Très visible dans l'identité gamification.

`src/components/user/profile/HESHouseQuiz.vue`

Important pour l'attribution / onboarding maison.

## Réflexe de reprise

Si tu n'as pas le temps de tout lire :

1. `layout/`
2. `common/forms/AuthForm.vue`
3. `admin/widgets/*`
4. `social/library/MainFeedSupabase.vue`
5. `events/EventForm.vue`
6. composants PFP directement depuis les vues / composants admin métier

## Note de dette technique

Le dossier composants contenait aussi quelques reliquats non runtime qui ne participaient pas à l'application active. Ils ont été retirés du repo pendant ce chantier pour réduire le bruit.

# Routing, navigation et contrôle d'accès

## Organisation

- `src/router/routes/*.js`: routes par domaine
- `src/router.js`: garde globale, `meta.need`, routes dynamiques

## Fichiers de routes

- `auth.js`
- `pages.js`
- `admin.js`
- `pfp.js`
- `social.js`
- `apps.js`

## Meta importante

- `requiresAuth`
- `need`
- `requiredRole`
- `requiresModuleOwnership`

## Point critique

Une partie des routes est chargée dynamiquement au runtime. Il faut donc diagnostiquer la navigation à la fois côté fichiers statiques et côté injection dynamique.

# Modules de routes et organisation du router

## NAVIGATION

Cette page documente l'organisation réelle de `src/router/routes/` et la logique de composition du routeur.

## Point d'entrée

Le routeur agrège les modules suivants depuis `src/router/routes/index.js` :

- `auth.js`
- `pages.js`
- `calendar.js`
- `media.js`
- `profiles.js`
- `admin.js`
- `pfp.js`
- `social.js`
- `users.js`
- `votations.js`
- `apps.js`

Le tableau final est injecté dans `createRouter()` depuis `src/router.js`.

## Rôle de chaque module

| Module                | Rôle principal                                                  |
|-----------------------|-----------------------------------------------------------------|
| <code>auth.js</code>  | login, register, reset password, pages d'accès                  |
| <code>admin.js</code> | dashboard, sécurité, planning, outils admin, gamification admin |

| Module    | Rôle principal                                                    |
|-----------|-------------------------------------------------------------------|
| pfp.js    | formation pratique, votations PFP, secrétariat, validation        |
| social.js | feed, hashtags, communautés                                       |
| users.js  | listes utilisateurs, formulaires, détails institutions et profils |
| apps.js   | applications intégrées, outils, mobile, capsules                  |

## auth.js

Contient notamment :

- /
- /home
- /register
- /reset-password
- /verification
- /access
- /auth-error

Ces routes servent de sas d'entrée et de gestion des erreurs d'authentification.

## admin.js

C'est un des modules les plus denses. Il couvre :

- dashboard principal et dashboards spécialisés ;
- sécurité et RBAC ;
- dashboards soins infirmiers ;
- outils académiques ;
- planning ;
- lists admin ;
- gamification admin ;
- route editor dynamique.

Exemples de routes importantes :

- `/admin`
- `/admin/supabase-diagnostic`
- `/admin/security/rbac`
- `/admin/planning/*`
- `/admin/gamification/*`

## **pfp.js**

Module très critique. Il porte :

- les anciennes routes historiques PFP ;
- les alias propres sous `/admin/formation-pratique/*` ;
- les vues secrétariat ;
- les campagnes de votation ;
- l'affectation et la validation.

Particularité importante :

- beaucoup de routes dépendent encore de `page1.access`.

## **social.js**

Routes sociales principales :

- `/feed`
- `/mention/:group`
- `/hashtag/:hashtag`
- `/communities`
- `/communities/:id`

Ces vues sont connectées aux stores sociaux et aux endpoints backend dédiés.

## **users.js**

Porte un mélange de :

- listes admin ;
- formulaires historiques ;
- pages de détail ;
- vues institutions.



Ce module est fonctionnel mais un peu hétérogène dans sa responsabilité.

## `apps.js`

Regroupe :

- chat
- mail
- notes
- files
- events
- tools
- rom-runner
- chatbot
- capsules insuffisance rénale
- routes mobiles spécifiques

Il contient aussi le catch-all `/:pathMatch(.*)*`.

## Conventions d'accès observées

Les modules utilisent principalement :

- `requiresAuth`
- `meta.need`
- plus rarement `requiredRole`

Le router normalise ensuite ces métadonnées dans `src/router.js`.

## Réflexes de reprise

Quand une route pose problème :

1. identifier son module source ;
2. vérifier son `meta.need` ;
3. vérifier son éventuel `requiredRole` ;
4. vérifier si elle est concernée par les routes dynamiques ;
5. vérifier ensuite la vue, le store et le service associés.

# Catalogue des routes frontend

## Objectif

Cette page sert de carte de navigation technique. Elle relie chaque famille de routes a ses vues principales.

## Regle de lecture

Le point d'entree du routing est:

- `src/router.js`
- `src/router/routes/index.js`

Les routes sont ensuite regroupees par domaines dans `src/router/routes/*.js`.

## Familles de routes

### `auth.js`

Vues principales:

- `auth/LoginHome.vue`
- `auth/RegisterView.vue`
- `pages/ResetPassword.vue`
- `auth/VerificationView.vue`
- `auth/LockScreenView.vue`
- `auth/AccessView.vue`
- `auth/AuthErrorView.vue`

Usage:

- authentication
- recovery
- erreurs et accès refusés

### `pages.js`

Vues principales:

- `home/MapView.vue`

- `institutions/Institution.vue`
- `institutions/PlaceListView.vue`
- `home/FaqView.vue`
- `home/TermsView.vue`
- `home/InfoExterneView.vue`
- `home/HistoryView.vue`
- `home/DocumentsView.vue`
- `home/PushView.vue`
- `home/RoleManagement.vue`
- `home/RouterView.vue`
- `home/PermissionsView.vue`
- `pages/FeedbackaStudentView.vue`
- `associations/AlpinPhysioView.vue`
- `documentation/PrimeVueDocsView.vue`

Usage:

- pages transverses et institutionnelles

## `admin.js`

Bloc le plus volumineux du routeur.

Sous-domaines couverts:

- dashboards admin
- role management
- planning admin
- soins infirmiers
- academic
- courses
- tools
- gamification admin

Exemples de vues:

- `admin/DashboardView.vue`
- `admin/DashboardRMView.vue`
- `admin/planning/WeeklyPlanningAdminView.vue`

- `admin/academic/AcademicKanbanView.vue`
- `admin/gamification/UserManagementView.vue`

## **pfp.js**

Bloc critique formation pratique.

Sous-domaines couverts:

- gestion PFP historique
- formation pratique physio
- secretariat
- votations
- affectations
- validations

Exemples de vues:

- `admin/pfp/ManagementOffreView.vue`
- `admin/pfp/PlacesAssignedView.vue`
- `admin/formation-pratique/EtudiantsViewPHYFP.vue`
- `admin/formation-pratique/VotationPFPViewPHYFP.vue`
- `admin/formation-pratique/secretariat/VueDEnsembleFP.vue`

## **calendar.js**

Vues principales:

- `planning/HomePlanning.vue`
- `planning/CalendrierFormationPlein.vue`
- `planning/CalendrierSemestriel.vue`
- `planning/CalendrierModule.vue`
- `planning/CalendrierEnseignant.vue`
- `planning/CalendarMyCourses.vue`
- `planning/CalendarMyModules.vue`

## **media.js**

Vues principales:

- `media/MediaHubPage.vue`
- `media/ModulesPage.vue`
- `media/ModuleVideosPage.vue`
- `media/VideoValidationPage.vue`
- `admin/ModuleAdminPage.vue`
- `media/VimeoTestPage.vue`

## **social.js**

Vues principales:

- `social/FeedView.vue`
- `social/MentionView.vue`
- `social/HashtagView.vue`
- `social/CommunitiesView.vue`
- `social/CommunityView.vue`
- `social/CommunityInfoView.vue`

## **apps.js**

Vues principales:

- `apps/tasklist/Index.vue`
- `apps/chat/IndexChat.vue`
- `apps/files/FilesView.vue`
- `apps/mail/Index.vue`
- `apps/notes/NotesWorkspaceView.vue`
- `apps/events/EventManagerView.vue`
- `apps/tools/ToolsView.vue`
- `apps/tools/ChatBotView.vue`
- `apps/rom-runner/RomRunnerView.vue`
- `pages/CareConvers.vue`
- `bloc capsulesInsuffiance/*`

## **users.js**

Vues principales:

- `admin/users/UserListView.vue`
- `admin/users/StudentListView.vue`

- `admin/users/TeacherListView.vue`
- `admin/institutions/InstitutionListView.vue`

## `votations.js`

Vues principales:

- `admin/votations/VotationGenericView.vue`
- `admin/votations/VotationManagementView.vue`
- `admin/places/PlaceAssignmentView.vue`
- `admin/validation/ValidationView.vue`
- `admin/validation/ReceptionView.vue`

## Regle de debug

Quand tu cherches une page:

1. trouve sa route dans `src/router/routes/*.js`
2. ouvre la vue correspondante
3. repere les stores et services importes
4. vérifie ensuite auth, rôles et source de données

# Stores, services et logique métier

## Separation de responsabilites

- vue: orchestration
- composant: rendu local
- store: etat partage
- service : logique métier
- composable: logique transverse

## Stores principaux

- `authStore.js`
- `userStore.js`
- `role.js`
- `institutionsStore.js`
- `placesStore.js`
- `postsStore.js`
- `votesStore.js`
- `eventStore.js`

## Services a connaitre

- `studentsService.js`
- `planningService.js`
- `rolesService.js`
- `resultatVotationService.js`
- `gamificationServiceSupabase.js`
- `notificationService.js`

# Stores critiques et responsabilités

## ÉTAT APPLICATIF

Cette page complète la cartographie stores/services en mettant l'accent sur les stores les plus structurants.

## Stores observés dans `src/stores/`

Extraits notables :

- `authStore.js`
- `role.js`
- `userStore.js`
- `placesStore.js`
- `praticiensStore.js`
- `praticiensFormateursStore.js`
- `postsStore.js`
- `chatStore.js`
- `votesStore.js`
- `documentStore.js`
- `eventStore.js`

### `authStore.js`

Responsabilité :

- gérer l'utilisateur courant ;
- supporter Firebase et Supabase ;
- restaurer la session ;
- surveiller l'expiration de session ;
- gérer le mode bypass auth.

Pourquoi il est critique :



- il conditionne tout le cycle d'accès ;
- il influence directement `router.beforeEach` ;
- il détermine le provider actif.

## **role.js**

Responsabilité :

- charger les permissions runtime ;
- exposer `can()` ;
- gérer `isSuper`.

Pourquoi il est critique :

- il traduit la session en permissions concrètes ;
- il mélange RPC et fallback `user_profiles` ;
- il porte une grande partie de la logique d'autorisation front.

## **userStore.js**

Responsabilité probable :

- maintenir le profil utilisateur enrichi ;
- gérer la lecture/abonnement des données liées à l'utilisateur.

Pourquoi il est critique :

- il sert souvent de pont entre session auth et données métier utilisateur.

## **placesStore.js**

Responsabilité probable :

- centraliser l'état des places PFP ;
- porter les listes et chargements liés aux campagnes.

Pourquoi il est critique :

- la PFP dépend fortement des places ;
- il peut être impliqué dans l'affectation, les validations et les suivis.

## **praticiensStore.js et praticiensFormateursStore.js**

Responsabilité probable :

- gérer les praticiens et praticiens formateurs ;
- alimenter les vues d'administration et les correspondances PFP.

Pourquoi ils sont critiques :

- ils structurent une partie du référentiel métier de la formation pratique.

## **postsStore.js**

Responsabilité probable :

- gestion du feed social ;
- chargement des posts, communautés, hashtags ou flux apparentés.

Pourquoi il est critique :

- dépendance au backend social ;
- impacts possibles sur l'engagement et la gamification.

## **votesStore.js**

Responsabilité probable :

- centraliser les données de votation ;
- porter certains écrans de résultats ou de preview.

Pourquoi il est critique :

- les votations sont un flux métier sensible ;
- l'état de ce store peut avoir un impact direct sur les décisions métier affichées.

## **Réflexes de lecture**

Pour un store critique, documenter systématiquement :

- sa source de données ;
- les services appelés ;

- les vues qui le consomment ;
- les permissions nécessaires ;
- les effets de bord sur d'autres domaines.

## Ordre de priorité de lecture recommandé

1. `authStore.js`
2. `role.js`
3. `userStore.js`
4. `placesStore.js`
5. `postsStore.js`
6. `votesStore.js`

# Cartographie stores et services

## Objectif

Cette page sert à retrouver rapidement quelle couche est touchée pour une fonctionnalité donnée :

- store Pinia
- service métier
- composable
- éventuel backend

## Règle de lecture

En pratique :

1. la vue orchestre
2. le store porte l'état partagé
3. le service porte la logique métier et les requêtes
4. le backend n'intervient que pour certains flux

## Stores critiques

| Store                           | Rôle principal                          | Dépendances directes                                                           |
|---------------------------------|-----------------------------------------|--------------------------------------------------------------------------------|
| <code>authStore.js</code>       | session multi-provider                  | Firebase Auth, Supabase Auth                                                   |
| <code>role.js</code>            | permissions runtime                     | Supabase RPC<br><code>api my permissions,</code><br><code>user_profiles</code> |
| <code>userStore.js</code>       | session Supabase +<br>profil temps réel | <code>user_profiles</code> , Realtime<br>Supabase                              |
| <code>placesStore.js</code>     | gestion des places                      | services PFP / tables places                                                   |
| <code>praticiensStore.js</code> | praticiens formateurs                   | flux PFP                                                                       |
| <code>postsStore.js</code>      | feed social                             | social backend / Supabase                                                      |

| Store                         | Rôle principal        | Dépendances directes         |
|-------------------------------|-----------------------|------------------------------|
| <code>eventStore.js</code>    | événements            | services événements          |
| <code>documentStore.js</code> | documents et fichiers | storage / services documents |
| <code>votesStore.js</code>    | votations             | services votation            |

## Trio auth / session / permissions

### `authStore.js`

Responsabilités :

- connexion Firebase
- connexion Supabase
- restauration de session
- refresh session
- monitoring session

Point critique :

- il décide du provider actif et influence toute la navigation

### `role.js`

Responsabilités :

- charger les permissions de l'utilisateur
- exposer `can()`
- gérer `isSuper`

Point critique :

- s'appuie d'abord sur la RPC `api_my_permissions`, puis fallback sur `user_profiles`

### `userStore.js`

Responsabilités :

- maintenir la session Supabase locale

- charger le profil `user_profiles`
- s'abonner aux mises à jour temps réel du profil

Point critique :

- plus centré sur le profil que sur l'auth globale

## Services critiques

| Service                                     | Rôle                                   |
|---------------------------------------------|----------------------------------------|
| <code>rolesService.js</code>                | compatibilité rôle Firebase / Supabase |
| <code>studentsService.js</code>             | logique étudiant / cohortes            |
| <code>planningService.js</code>             | logique planning                       |
| <code>academicYearService.js</code>         | année académique                       |
| <code>resultatVotationService.js</code>     | résultats de votation                  |
| <code>pfpAlertsService.js</code>            | alertes PFP                            |
| <code>gamificationServiceSupabase.js</code> | XP, maisons, stats, classement         |
| <code>notificationService.js</code>         | notifications                          |
| <code>videoLibraryService.js</code>         | bibliothèque vidéo                     |
| <code>supabaseStorageService.js</code>      | stockage Supabase                      |

## Exemple de chaînes typiques

### Login et droits

```
Vue login
→ authStore
→ router.js
→ roleStore
→ RPC Supabase / user_profiles
```

## Feed social

```
Vue social
→ postsStore
→ service / backend social
→ posts / hashtags / communities
```

## Formation pratique

```
Vue admin PFP
→ placesStore / praticiensStore / votesStore
→ services PFP
→ Supabase / backend / scripts selon le flux
```

## Règle de modification

Si tu modifies :

- l'état partagé : store
- les appels données : service
- la logique d'accès : authStore, roleStore, router
- l'effet visuel local : vue ou composant

# Catalogue des services métier

## Objectif

Donner une vue d'ensemble du dossier `src/service/` pour savoir où chercher selon le problème.

## Services par grandes familles

### Auth, rôles et profils

- `rolesService.js`
- `userProfileAutoCreation.js`
- `notificationService.js`

### Planning, académique et charge

- `planningService.js`
- `academicPlanningService.js`
- `academicYearService.js`
- `modulePlanningService.js`
- `moduleService.js`
- `modulesService.js`
- `workloadService.js`
- `ticketService.js`

### Dashboards et KPI

- `dashboardService.js`
- `dashboardSupabaseService.js`
- `dashboardQuickStatsService.js`
- `adminDashboardService.js`
- `adminKpiService.js`
- `academicKpiService.js`
- `enseignantDashboardService.js`
- `rmDashboardService.js`

### Formation pratique et votations

- `studentsService.js`



- resultatVotationService.js
- votationSessionService.js
- votesBackendService.js
- pfpAlertsService.js
- pfpStatsService.js
- intelligentAlertsService.js
- periodComparisonService.js
- dataQualityService.js

## Gamification

- gamificationServiceSupabase.js
- gamificationService.js
- gamificationIntegration.js
- badgesService.js
- challengesService.js
- questsService.js
- questExpirationService.js
- userQuestsService.js
- adminQuestsService.js
- hesHousesService.js
- defisService.js

## Media et contenus

- mediaService.js
- institutionMediaService.js
- videoLibraryService.js
- vimeoService.js
- notesService.js

## Stockage et données

- supabaseStorageService.js

## Intégrations et divers

- githubService.js
- EventService.js

## Services à connaître absolument

### `rolesService.js`

Pont entre le monde Firebase legacy et le monde Supabase pour les rôles.

### `planningService.js`

Service central pour la logique de planning côté métier.

### `studentsService.js`

Service structurant du domaine étudiants / PFP.

### `resultatVotationService.js`

Service critique pour les résultats et calculs liés aux votations.

### `gamificationServiceSupabase.js`

Service dense, avec :

- lecture utilisateur
- maisons
- calculs de niveaux
- classement des maisons
- cache mémoire

## Règle de modification

Quand une fonctionnalité devient importante :

- la vue ne doit pas porter le calcul principal
- le service doit devenir la source de logique
- le store doit rester centré sur l'état et l'orchestration

# Matrice vue → store → service → backend → données

## TRAÇABILITÉ

Cette matrice sert à comprendre où se situe la logique réelle : vue, store, service, endpoint, table ou RPC.

## Règle de lecture

Pour une fonctionnalité donnée :

1. commencer par la vue ;
2. repérer le store ou le service appelé ;
3. vérifier si l'appel part vers Supabase direct ou `/api/*` ;
4. identifier enfin la table, la RPC ou le module backend concerné.

## Matrice synthétique

| Domaine / écran    | Vue principale                                             | Store / s                                                  |
|--------------------|------------------------------------------------------------|------------------------------------------------------------|
| Feed social        | <code>views/social/FeedView.vue</code>                     | <code>postsStore.js</code>                                 |
| Communautés        | <code>views/social/CommunitiesView.vue</code>              | accès Supabas<br>backend social                            |
| Feedbacka étudiant | <code>views/pages/FeedbackaStudentView.vue</code>          | <code>feedbackaStore</code>                                |
| Votation PFP       | <code>views/admin/votations/VotationGenericView.vue</code> | <code>resultatVotati</code><br><code>votesBackendSe</code> |

| Domaine / écran     | Vue principale                                                        | Store / service                    |
|---------------------|-----------------------------------------------------------------------|------------------------------------|
| Places assignées    | <code>views/admin/pfp/PlacesAssignedView.vue</code>                   | logique vue + service              |
| Institutions PFP    | <code>views/admin/formation-pratique/InstitutionsViewPHYFP.vue</code> | vues + service                     |
| Médias institution  | vues institution/admin                                                | <code>institutionMediaStore</code> |
| RBAC admin          | <code>views/admin/security/RBACAdminView.vue</code>                   | accès direct Supabase              |
| Routes dynamiques   | <code>views/home/DynamicRoutesEditorView.vue</code>                   | vue + Supabase                     |
| Diagnostic Supabase | <code>views/admin/SupabaseDiagnosticView.vue</code>                   | vue                                |

## Flux 1 — Social

Chaîne principale observée :

```
FeedView / composants sociaux
→ postsStore.js
→ axios vers /api/posts
→ backend/supabase/postsBackendStore.js
→ Supabase (posts, post_media, hashtags, RPC create_post_with_hashtags)
```

Points importants :

- le social n'est pas purement frontend ;
- le backend sert d'orchestrateur pour certains flux d'écriture ;
- certaines vues communautés lisent aussi directement Supabase.

## Flux 2 — Feedbacks

Chaîne principale observée :

```
FeedbackaStudentView / outils admin Feedbacka
→ feedbackaStore.js
→ /api/feedbacka
→ backend/supabase/feedbackaBackend.js
→ stockage backend / logique d'évaluation
```

Points importants :

- store orienté API HTTP ;
- endpoints séparés pour test, submit et submissions ;
- domaine plus "outil métier" que simple CRUD.

## Flux 3 — Votation et résultats PFP

Chaîne principale observée :

```
VotationGenericView / vues PFP
→ resultatVotationService.js
→ /api/resultat-votation/*
→ backend/supabase/resultatVotationStoreBackend.js
→ tables résultats + RPC get_algorithm_results / get_student_result
```

Tables / mécanismes observés :

- `student_result_vote`
- `vote_place_aggregation`
- RPC `get_student_result`
- RPC et logique d'algorithme côté backend

Point important :

- ce domaine alterne entre appels backend et lectures Supabase directes ;
- il faut distinguer lecture simple, simulation, génération de propositions et confirmation.

## Flux 4 — Institutions et médias

Chaîne observée :

```
Vue institution / admin institution
→ institutionMediaService.js
→ /api/institutions/:id/images
→ backend institutionsStoreBackend
→ Supabase Storage bucket institutions
```

Point important :

- upload/suppression passent par backend ;
- les URLs images sont ensuite réinterprétées en chemin storage.

## Flux 5 — Sécurité et rôles

Chaîne observée :

```
RBACAdminView / ManageUserRoles / router
→ roleStore + vues admin sécurité
→ Supabase direct
→ permissions / roles / role_permissions / user_roles
→ RPC update_user_permissions, get_user_permissions
```

Point important :

- une partie de la sécurité vit en lecture/écriture directe base ;
- le router et `roleStore` dépendent ensuite de ce socle.

## Réflexe de maintenance

Si une fonctionnalité casse :

- vérifier d'abord si elle est backend-driven ou Supabase-direct ;
- vérifier si la logique est dans la vue, le store ou un service ;
- vérifier si la permission échoue avant même l'appel ;
- vérifier enfin si la donnée source a changé.

# Matrice route → vue → données

## TRAÇABILITÉ NAVIGATION

Cette page sert à partir d'une route concrète pour retrouver la vue, la couche logique et la donnée touchée.

### Routes critiques sélectionnées

| Route                            | Vue                                                   |
|----------------------------------|-------------------------------------------------------|
| /feed                            | views/social/FeedView.vue                             |
| /communities                     | views/social/CommunitiesView.vue                      |
| /admin/supabase-diagnostic       | views/admin/SupabaseDiagnosticView.vue                |
| /admin/security/rbac             | views/admin/security/RBACAdminView.vue                |
| /admin/routes-editor             | views/home/DynamicRoutesEditorView.vue                |
| /management_votation_prioritaire | views/admin/pfp/ManagementVotationPrioritaireView.vue |
| /places_asssigned                | views/admin/pfp/PlacesAssignedView.vue                |
| /votation/:pfpType               | views/admin/votations/VotationGenericView.vue         |

| Route                                                           | Vue                                |
|-----------------------------------------------------------------|------------------------------------|
| <code>/admin/formation-pratique/secretariat/vue-ensemble</code> | <code>VueDEnsembleFP.vue</code>    |
| <code>/admin/tools/feedbacka/*</code>                           | vues Feedbacka                     |
| <code>/tools/ftp-upload</code>                                  | <code>FTPUploadTestView.vue</code> |

## Lecture par familles de routes

### Routes sociales

- proviennent de `social.js`
- s'appuient sur le store posts ou des lectures Supabase directes
- touchent `posts`, `communities`, `user_communities`, `hashtags`

### Routes admin sécurité

- proviennent de `admin.js`
- pilotent la sécurité applicative
- touchent surtout les tables RBAC et `user_profiles`

### Routes PFP historiques

- proviennent de `pfp.js`
- utilisent fortement `page1.access`
- peuvent mélanger backend, Supabase direct et conventions legacy

### Routes votations

- proviennent de `votations.js`
- s'appuient sur `resultatVotationService.js`
- finissent souvent dans `student_result_vote` ou dans des RPC dédiées

### Routes outils / apps

- proviennent de `apps.js`



- couvrent un spectre plus large ;
- certaines sont simples, d'autres dépendent du backend ou de services externes.

## Réflexe de diagnostic

Quand une URL pose problème :

1. retrouver son module de route ;
2. ouvrir la vue ;
3. chercher le store/service utilisé ;
4. identifier si l'appel est direct Supabase ou `/api/*` ;
5. vérifier la table, la RPC ou le backend cible.

# UI, layout et style applicatif

## Stack UI

- PrimeVue
- PrimeFlex
- SCSS
- theme personnalisée

## Shell principal

- `src/layout/AppLayout.vue`
- `src/layout/AppSidebar.vue`
- `src/layout/AppTopbar.vue`
- `src/layout/AppMenu.vue`

## Styles structurants

- `src/assets/styles/styles.scss`
- `src/assets/styles/mobile-scale.css`
- `src/assets/theme/`
- `public/layout/styles/theme/`

# PWA, service worker et cache

## Ce qui existe

Le frontend est prepare comme PWA via Vite.

## Point central

Le comportement le plus important est actuellement dans `src/main.js`:

- comparaison de `APP_VERSION`
- purge des anciens service workers
- suppression des caches
- rechargement de l'application si la version change

## Implication

Une modification du cycle de build ou du versionnement peut impacter:

- les utilisateurs deja connectes
- la fraicheur des assets
- le comportement offline

## Regle

Ne pas toucher a la gestion de version du service worker sans tester un vrai cycle de mise a jour navigateur.

# Authentification, rôles et permissions

## Résumé

Le projet n'a pas un seul mécanisme d'accès. Il combine :

- Firebase Auth
- Supabase Auth
- contrôle front par route
- contrôle métier par permissions
- contrôle base par RLS

## Fichiers de référence

- `src/firebase.js`
- `src/supabase.js`
- `src/router.js`
- `src/stores/authStore.js`
- `src/stores/role.js`
- `src/service/rolesService.js`

## Ce que fait Firebase

- authentication legacy
- source encore active dans certains flux utilisateurs
- intégration avec Realtime Database et Storage

## Ce que fait Supabase

- authentification moderne
- persistance de session
- PKCE flow
- base relationnelle
- permissions et policies

## Contrôle d'accès en pratique

### Niveau 1 : session

L'utilisateur doit être connu via `authStore`.

## Niveau 2 : routing

Le router évalue :

- `requiresAuth`
- `meta.need`
- `requiredRole`

## Niveau 3 : rôles et permissions

Le `roleStore` expose la lecture des permissions via `can()`.

## Niveau 4 : base

Les policies RLS Supabase doivent encore autoriser les requêtes.

## Conséquence de maintenance

Un problème d'accès peut venir :

- du provider d'auth
- du router
- du rôle
- d'une policy RLS

Il faut donc vérifier toute la chaîne, pas seulement la page visible.

## Recommandations

- toute nouvelle page sensible doit définir explicitement `meta.need`
- toute nouvelle table sensible doit avoir un plan RLS
- toute correction de droit doit être testée avec un vrai profil non-admin

# Cycle auth, rôles et routing

## CYCLE RÉEL

Cette page synthétise le comportement observé dans `src/router.js`, `authStore.js` et `role.js`.

## Vue d'ensemble

Trois briques décident de l'accès :

- `authStore` : connaît l'utilisateur et le provider actif ;
- `roleStore` : charge les permissions ;
- `router.beforeEach` : applique les règles d'accès page par page.

## Séquence de navigation

```
Navigation
→ chargement éventuel des routes dynamiques
→ checkAuthState()
→ roleStore.init()
→ lecture de meta.need
→ lecture éventuelle de meta.requiredRole
→ redirection / accès / refus
```

## Ce que fait `authStore`

Dans `src/stores/authStore.js`, le store :

- gère Firebase et Supabase ;
- sait si l'app est en mode bypass auth ;
- restaure la session ;
- surveille l'expiration de session Supabase ;
- écoute les événements auth des deux providers.

## Cas importants

- `VITE_DISABLE_AUTH=true` force un utilisateur invité ;
- Firebase reste prioritaire si sa session est restaurée en premier ;
- sinon Supabase devient le provider actif ;
- le store essaie un refresh si le token Supabase est expiré ou invalide.

## Ce que fait `roleStore`

Dans `src/stores/role.js`, le store :

- récupère la session Supabase courante ;
- charge les permissions via RPC `api_my_permissions` ;
- fallback sur `user_profiles.role` et `user_profiles.permissions` ;
- expose `can()` et `isSuper`.

## Source de vérité des permissions

Ordre observé :

1. RPC Supabase `api_my_permissions`
2. fallback `user_profiles.role`
3. fallback `user_profiles.permissions`

## Cas important

La permission `super.all` court-circuite les contrôles métier front.

## Ce que fait le router

Dans `src/router.js`, le guard :

1. normalise `meta.need` sur toutes les routes ;
2. charge une fois les routes dynamiques ;
3. exécute `checkAuthState()` ;
4. initialise `roleStore` ;
5. applique les redirections et permissions.

## Normalisation de `meta.need`

Si une route n'a pas de `meta.need` :

- `requiresAuth=true` devient `authenticated`

- sinon la route devient `public`

Cela évite qu'une route protégée reste sans besoin explicite.

## Cas de redirection importants

### Route `/`

Comportement observé :

- utilisateur connecté → redirection `/feed`
- sinon → redirection `/home`

### Routes password reset

Les routes `/reset-password` et `/new-password` sont laissées passer tôt pour éviter de consommer le code PKCE.

### Route refusée

Si une permission manque, redirection vers `/access`.

### `need` versus `requiredRole`

Le projet utilise deux mécanismes :

- `meta.need` : mécanisme recommandé et le plus lisible ;
- `meta.requiredRole` : mécanisme historique encore présent.

Le guard supporte les deux. Il faut donc vérifier les deux lors d'un diagnostic.

## Pièges de maintenance

- une route peut sembler publique si `meta.need` a été mal renseigné ;
- un utilisateur peut être connecté mais sans permissions chargées ;
- une page peut passer le front mais échouer côté RLS ;
- le mode bypass auth peut masquer un vrai problème de droits ;
- des routes dynamiques peuvent modifier le résultat attendu après chargement.

## Fichiers à lire ensemble

- `src/router.js`



- `src/router/routes/index.js`
- `src/stores/authStore.js`
- `src/stores/role.js`
- `src/service/rolesService.js`

## Réflexe de diagnostic

Quand une page est inaccessible :

1. vérifier la session ;
2. vérifier `authProvider` ;
3. vérifier `roleStore.perms` ;
4. vérifier `meta.need` ;
5. vérifier `requiredRole` si présent ;
6. vérifier ensuite la policy RLS ou l'API backend.

# Modèle de permissions et matrice d'accès

## Niveaux du modèle

Le contrôle d'accès se fait par superposition :

1. provider d'auth
2. session utilisateur
3. route front
4. permission chargée par `roleStore`
5. enforcement base via RLS

## Sources de permission

### Source principale

- RPC Supabase `api_my_permissions`

### Fallback

- lecture de `user_profiles.role`
- lecture de `user_profiles.permissions`

### Héritage legacy

- `requiredRole`
- rôles Firebase historiques
- `rolesService.js`

## Clés d'accès visibles dans le code

| Type             | Exemples                                                                  |
|------------------|---------------------------------------------------------------------------|
| Spéciales        | <code>public</code> , <code>anonymous</code> , <code>authenticated</code> |
| Super permission | <code>super.all</code>                                                    |
| Admin            | <code>admin</code> , <code>AdminPhysio</code>                             |

| Type               | Exemples                                                                                 |
|--------------------|------------------------------------------------------------------------------------------|
| Académique / soins | <code>RMSoins</code> , <code>EnseignantSoins</code> , <code>PlanificateurHoraires</code> |
| Formation pratique | <code>page1.access</code>                                                                |
| Édition            | <code>editor</code>                                                                      |

## Matrice de lecture

| Cas                     | Session requise | Permission front                              | RLS à vérifier     |
|-------------------------|-----------------|-----------------------------------------------|--------------------|
| Page publique           | non             | <code>public</code> ou <code>anonymous</code> | rarement           |
| Page connectée standard | oui             | <code>authenticated</code>                    | oui selon données  |
| Page admin              | oui             | <code>admin</code> ou équivalent              | oui                |
| Page PFP historique     | oui             | souvent <code>page1.access</code>             | oui                |
| Action critique backend | oui             | front + backend                               | oui ou service key |

## Cas particulier `super.a11`

`roleStore.isSuper` court-circuite une partie des contrôles front. Cela ne dispense pas de penser la sécurité côté base et côté backend.

## Point faible structurel à connaître

Le modèle n'est pas entièrement uniformisé :

- une partie des routes utilise `need`
- une partie conserve `requiredRole`
- une partie de la logique métier garde des conventions legacy

Donc toute évolution permissionnelle doit être testée de bout en bout.

## Recommandation de convergence

Pour les futures évolutions :

- privilégier `meta.need`
- centraliser la logique de lecture dans `roleStore.can()`
- documenter toute nouvelle permission métier
- limiter le recours aux conventions implicites

# Sécurité, rôles et compatibilité legacy

## COMPATIBILITÉ HISTORIQUE

Cette page détaille le rôle de `rolesService.js` et les raisons de la coexistence Firebase / Supabase.

### `rolesService.js`

Le service porte un rôle de compatibilité entre plusieurs modèles :

- rôles Firebase historiques ;
- rôles Supabase modernes ;
- format objet legacy du type `{ admin: true } ;`
- lecture d'un rôle unique dans `user_profiles`.

## Sources gérées

### Côté Firebase

- lecture via Realtime Database ;
- chemins legacy du type `Users/{userId}/Roles`.

### Côté Supabase

Le service essaie plusieurs stratégies :

1. `user_metadata.roles`
2. table `user_roles`
3. table `user_profiles.role`
4. fallback utilisateur simple

## Pourquoi c'est important

Le router et certains écrans anciens peuvent encore dépendre :

- d'un schéma objet legacy ;

- d'un rôle unique ;
- ou de permissions plus modernes.

## Risques de maintenance

- croire qu'un seul mécanisme de rôle existe ;
- modifier uniquement `user_roles` alors que certains écrans lisent encore `user_profiles.role` ;
- casser une compatibilité Firebase encore utile dans certains flux.

## Réflexe de reprise

Quand un droit semble incohérent :

1. vérifier `roleStore` ;
2. vérifier `rolesService` ;
3. vérifier `user_profiles.role` ;
4. vérifier `user_roles` ;
5. vérifier s'il reste un chemin Firebase legacy impliqué.

# Rôles & Permissions

Ce document décrit le modèle de permissions consommé par le front et son usage dans le routeur.

## Source des permissions (front)

- Store: `src/stores/role.js` (`useRoleStore`)
  - `perms: string[]` — liste des permissions
  - `isSuper` — vrai si `perms` contient `super.all`
  - `can(perm | perm[])` — vérifie une ou plusieurs permissions
  - Initialisation: `init()` charge la session Supabase et les permissions via une RPC (par ex. `get_user_permissions` ou `api_my_permissions` selon votre configuration)

L'implémentation de la fonction RPC côté base doit renvoyer une liste de permissions pour l'utilisateur courant. Voir les migrations SQL pour la création de cette RPC.

## RPC attendue côté DB

- Le store appelle par défaut `supabase.rpc('api_my_permissions')`.
- Si votre schéma expose déjà `get_user_permissions(uid text)`, créez un alias compatible:

```
-- Alias simple qui transforme le tableau en table (colonne perm)
create or replace function public.api_my_permissions()
returns table(perm text)
language sql
security definer
as $$
    select unnest(public.get_user_permissions(auth.uid()::text)) as perm;
$$;
```

- Mise à jour des permissions (administration):
  - `update_user_permissions(target_user_id text, new_permissions text[])`.
    - Définie dans `supabase_migrations/add_permissions_to_user_profiles.sql`.
    - En cas de cache PostgREST, voir `supabase_migrations/reload_permissions_functions.sql` (NOTIFY pgrst,

'reload schema').

## Dans le routeur

- `need: 'page1.access'`
- `need: ['admin', 'editor']`
- Spéciaux: `public`, `anonymous`, `authenticated`
- `meta.requiredRole`: support historique basé sur des rôles nommés
- Bypass: `isSuper` accorde l'accès quelle que soit la permission

## Convention de nommage (conseillée)

- Préfixer par domaine:
  - `admin.*` (ex: `admin.settings`, `admin.users`)
  - `page*.access` (ex: `page1.access`, `page2.access`)
  - `media.*`, `fp.*` (formation pratique), `si.*` (soins-infirmiers)

## Exemples

```
import { useRoleStore } from '@stores/role'
const roleStore = useRoleStore()
if (roleStore.can('page1.access')) {
  // afficher bouton ou lien
}
if (roleStore.can(['admin', 'editor'])) {
  // au moins une des permissions
}
```

## Bonnes pratiques

- Centraliser la logique d'autorisation avec `roleStore.can()`
- Préférer les permissions granulaires aux rôles macro
- Documenter les permissions métiers dans les specs (qui peut faire quoi)



# Route guards

## Point d'entrée

- `src/router.js`

## Ce que fait le guard global

1. charge les routes dynamiques
2. vérifie l'état de session
3. initialise le `roleStore`
4. évalue `meta.need`
5. gère `requiredRole`
6. applique les redirections

## Métadonnées importantes

- `requiresAuth`
- `need`
- `requiredRole`
- `requiresModuleOwnership`

## Point important

Le projet privilégie `meta.need` pour la lecture moderne des permissions.

# Supabase RLS

## Principe

RLS protège les lignes accessibles selon l'utilisateur et son contexte d'authentification.

## Règle pratique

Toute table sensible doit avoir des politiques explicites, pas implicites.

## Zones à surveiller

- profils
- institutions
- places
- données de vote
- documents

## Point critique du projet

Le front charge les permissions via `api_my_permissions`, mais cela ne remplace pas les politiques RLS sur les tables métier.

# Données, coexistence Firebase et Supabase

## Résumé

Le projet est en migration. Pour chaque fonctionnalité, il faut identifier si la source de vérité est Firebase, Supabase ou le backend Node.

## Réalité actuelle

Le système n'est pas encore totalement unifié :

- certaines données vivent encore côté Firebase
- d'autres sont déjà portées par Supabase
- certains flux passent par le backend

## Réflexe de reprise

Avant toute modification :

1. identifier la source de données réelle
2. vérifier les migrations et policies
3. vérifier si le frontend appelle un service direct ou un endpoint backend

# Tables critiques et responsabilités métier

## DONNÉES CRITIQUES

Cette page ne remplace pas le schéma complet, mais identifie les tables qui structurent réellement les flux métier du projet.

## Auth, profils et permissions

### user\_profiles

Rôle :

- profil enrichi utilisateur ;
- fallback de permissions ;
- rôle métier visible côté front.

Impact :

- auth ;
- permissions ;
- administration ;
- lecture des profils dans plusieurs domaines.

### roles

Rôle :

- catalogue des rôles du modèle RBAC.

### permissions

Rôle :

- catalogue des permissions attribuables.

### role\_permissions

Rôle :

- mapping rôle → permissions.

### **user\_roles**

Rôle :

- mapping utilisateur → rôles.

## **Formation pratique et votation**

### **StudentsPhysio**

Rôle :

- référentiel étudiant très utilisé dans les flux PFP ;
- suivi des stages validés ;
- historique et flags métier.

Champs métier fréquemment impliqués :

- pfp\_valided
- pfp2\_data
- sae
- cas\_particulier
- repondant\_hes

### **places**

Rôle :

- référentiel des places ;
- support des campagnes PFP ;
- capacité, critères, correspondance institutionnelle.

### **institutions**

Rôle :

- référentiel des institutions ;
- rattachement des places ;

- support des vues map et secrétariat.

### **praticiens\_formateurs**

Rôle :

- référentiel praticiens ;
- rattachement métier à certaines affectations ou vues.

### **student\_result\_vote**

Rôle :

- table centrale des résultats d'attribution ;
- stockage des assignations, statuts et sorties algorithmiques.

Pourquoi elle est critique :

- elle relie les campagnes, les assignations, les validations et une partie du suivi.

### **votation\_sessions**

Rôle :

- sessions de votation ;
- stockage de propositions PFP3/PFP4 ;
- gestion de priorités, drafts et états de campagne.

### **suivi\_cas\_particuliers**

Rôle :

- suivi manuel de cas spécifiques ;
- appui aux vues secrétariat et alertes.

### **recap\_cpt\_evaluation**

Rôle :

- agrégats / récapitulatifs CPT.

## **Social et engagement**

### **posts**

Rôle :

- base du feed social.

### **post\_media**

Rôle :

- médias rattachés aux posts.

### **hashtags**

Rôle :

- taxonomie hashtags.

### **communities**

Rôle :

- communautés sociales.

### **user\_communities**

Rôle :

- rattachement utilisateur ↔ communauté.

## **Planning et exploitation**

### **modules**

Rôle :

- structure pédagogique ;
- dépendances du planning et des modules.

### **calendar\_cells**

Rôle :

- cellules ou granularité de planning dans certaines vues.

# Stockage et médias

## Buckets storage observés

- institutions
- avatars
- documents étudiants

Le repo montre aussi des migrations dédiées à l'activation des buckets storage.

## Réflexe de reprise

Avant toute modification critique :

1. identifier la table maître du flux ;
2. identifier les tables secondaires associées ;
3. vérifier les migrations récentes ;
4. vérifier les politiques RLS ;
5. vérifier le ou les écrans qui lisent directement la table.



# RPC Supabase et surface SQL exposée

## SURFACE SQL

Cette page liste les RPC explicitement repérées dans le code et les migrations les plus significatives qui les encadrent.

## RPC observées côté frontend ou backend

### Permissions / rôles

- `api_my_permissions`
- `get_user_permissions`
- `update_user_permissions`

Usage :

- chargement des permissions runtime ;
- administration des rôles ;
- fallback de sécurité front.

### Votation / résultats

- `get_student_vote`
- `upsert_student_vote`
- `delete_student_vote`
- `has_student_voted`
- `get_all_student_votes`
- `count_votes`
- `get_top_voted_places`
- `get_algorithm_results`
- `get_student_result`

Usage :

- vote étudiant ;

- lecture de résultats ;
- statistiques ;
- calculs d'agrégats.

## Administration / maintenance

- `delete_user`
- `assign_quest_to_all_users`

## Migrations particulièrement visibles

Fichiers présents dans `supabase/migrations/` :

- `20260114_create_api_my_permissions.sql`
- `20260223_add_priority_to_votation_sessions.sql`
- `20260225_add_priority_reasons_and_draft.sql`
- `20260325_add_remarques_to_student_result_vote.sql`
- `20260529_copy_previous_year_place_propositions.sql`
- `20260609_enable_institutions_storage.sql`
- `20260610_enable_avatars_storage.sql`
- `20260610_enable_student_documents_storage.sql`
- `20260624_create_institution_offer_tracking.sql`

## Ce que cela veut dire pour la reprise

Le comportement métier ne vit pas uniquement :

- dans les vues ;
- dans les stores ;
- ou dans le backend Express.

Une partie importante du système vit aussi :

- dans les RPC Supabase ;
- dans les politiques RLS ;
- dans les migrations qui ajoutent colonnes, drafts, raisons, storage et règles métier.

## Risques de modification

- casser un flux en modifiant seulement le frontend ;
- ignorer une RPC utilisée indirectement par un service ;
- oublier qu'une migration ajoute un champ désormais attendu ;
- modifier une table sans vérifier la logique SQL associée.

## Réflexe d'audit

Avant de toucher un flux sensible :

1. chercher les appels `supabase.rpc(...)` ;
2. chercher les appels `supabase.from(...)` sur la table concernée ;
3. lire les migrations récentes liées au domaine ;
4. vérifier si un endpoint backend encapsule en plus une logique métier.

# Schéma Supabase

Vue d'ensemble du schéma de données côté Supabase/Postgres.

## Tables principales

- `public.user_profiles`
  - `id UUID` (PK)
  - `user_id TEXT` (UID auth; unique)
  - `email TEXT`
  - `role TEXT` (enum logique: admin, user, teacher, ...)
  - `house TEXT` (harmonis|elaris|doloris|solencia)
  - `permissions TEXT[]` (ajoutée par `add_permissions_to_user_profiles.sql`)
  - Index: `idx_user_profiles_user_id`, `idx_user_profiles_email`, `idx_user_profiles_role`, `idx_user_profiles_house`, `idx_user_profiles_permissions` (GIN)
- `public.gamification_data`
  - `user_id TEXT` (unique), `points INT`, `level INT`, `badges JSONB`, ...
  - Index: `idx_gamification_user_id`, `idx_gamification_house`, `idx_gamification_points`
- `public.institutions`
  - `InstitutionId TEXT` (PK), `Name TEXT`, `Canton TEXT`, `Latitude NUMERIC`, `Longitude NUMERIC`, `ImageURL JSONB`, ...
  - Triggers: update auto `UpdatedAt`
  - RLS: activée, politiques de lecture et `service_role` pour écritures (par défaut)
- `public.places`
  - `PlaceId TEXT` (PK), `InstitutionId TEXT`, `NomPlace TEXT`, `fileURL TEXT`
  - Spécialités (bool): `MSQ`, `SYSINT`, `AIGU`, `REHAB`, `AMBU`, `NEUROGER`
  - Langues (bool): `FR`, `DE`, `IT`, `ENG`
  - Capacités (JSONB): `PFP1A`, `PFP1B`, `PFP2`, `PFP3`, `PFP4`
  - `Remarques JSONB`, `praticiensFormateurs TEXT[]`
  - Index: par institution, nom, canton; GIN sur JSONB, composites sur spécialités/langues
  - RLS: activée; lecture authenticated/anon (exemples) et write `service_role`

## Relations

- `places.InstitutionId` → `institutions.InstitutionId` (FK conceptuelle; définir un FK explicite si requis)
- `gamification_data.user_id` ↔ `user_profiles.user_id`

## RLS (Row-Level Security)

- Activée sur `user_profiles`, `gamification_data`, `institutions`, `places`
- Politiques d'exemple fournies dans les migrations; à adapter selon les besoins métiers

## Fonctions & vues utiles

- Triggers d'`updated_at`
- `handle_new_user()` (création profil/gamification à l'inscription)
- Vues: `user_profiles_complete`
- RPC permissions: `get_user_permissions(uid TEXT)` (cf. `add_permissions_to_user_profiles.sql`)

## Exemples de requêtes

```
-- Institutions avec coordonnées manquantes
select "InstitutionId", "Name" from public.institutions where "Latitude" is
null or "Longitude" is null;
-- Places REHAB en FR ou DE
select "PlaceId", "NomPlace", "Canton"
from public.places
where "REHAB" is true and ("FR" is true or "DE" is true)
order by "NomPlace";
-- Permissions d'un utilisateur
select get_user_permissions('USER_UID');
```

# Catalogue des migrations Supabase

## Objectif

Donner une vue lisible des migrations presentes dans `supabase/migrations/`.

## Migrations repertoriees

| Fichier                                                        | Intention apparente                   |
|----------------------------------------------------------------|---------------------------------------|
| <code>20251215_create_feedbacka_module.sql</code>              | creation module Feedbacka             |
| <code>20260114_create_api_my_permissions.sql</code>            | creation RPC permissions              |
| <code>20260130_add_async_periods_to_planning.sql</code>        | periodes asynchrones planning         |
| <code>20260130_add_multiple_classes_support.sql</code>         | support classes multiples             |
| <code>20260130_clean_and_standardize_class_codes.sql</code>    | normalisation codes classes           |
| <code>20260130_fix_unique_constraint.sql</code>                | correction contrainte d'unicite       |
| <code>20260223_add_priority_to_votation_sessions.sql</code>    | priorite sur sessions de votation     |
| <code>20260225_add_priority_reasons_and_draft.sql</code>       | raisons de priorite et brouillon      |
| <code>20260316_add_student_count_to_classes.sql</code>         | nombre d'etudiants par classe         |
| <code>20260325_add_remarques_to_student_result_vote.sql</code> | remarques sur resultats vote etudiant |
| <code>20260422_grant_access_batch2_profiles.sql</code>         | grants profils batch 2                |
| <code>20260422_grant_access_to_physio_profiles.sql</code>      | grants profils physio                 |

| Fichier                                            | Intention apparente                   |
|----------------------------------------------------|---------------------------------------|
| 20260529_copy_previous_year_place_propositions.sql | copie propositions année précédente   |
| 20260609_enable_institutions_storage.sql           | storage institutions                  |
| 20260610_add_is_hidden_to_institutions.sql         | drapeau visibilité institutions       |
| 20260610_enable_avatars_storage.sql                | storage avatars                       |
| 20260610_enable_student_documents_storage.sql      | storage documents étudiants           |
| 20260610_fix_storage_grants_self_hosted.sql        | correction grants storage self-hosted |
| 20260624_create_institution_offer_tracking.sql     | suivi des offres institutions         |

## Migrations structurantes à connaître

### Permissions

- 20260114\_create\_api\_my\_permissions.sql

Impact:

- chargée par `roleStore`
- critique pour la lecture des permissions runtime

### Planning

- 20260130\_add\_async\_periods\_to\_planning.sql
- 20260130\_add\_multiple\_classes\_support.sql
- 20260130\_clean\_and\_standardize\_class\_codes.sql

Impact:

- domaine planning
- classes

- periodes

## Votation / PFP

- 20260223\_add\_priority\_to\_votation\_sessions.sql
- 20260225\_add\_priority\_reasons\_and\_draft.sql
- 20260325\_add\_remarques\_to\_student\_result\_vote.sql
- 20260529\_copy\_previous\_year\_place\_propositions.sql
- 20260624\_create\_institution\_offer\_tracking.sql

Impact:

- campagnes
- secretariat
- suivi des offres
- resultats

## Storage

- 20260609\_enable\_institutions\_storage.sql
- 20260610\_enable\_avatars\_storage.sql
- 20260610\_enable\_student\_documents\_storage.sql
- 20260610\_fix\_storage\_grants\_self\_hosted.sql

Impact:

- fichiers
- avatars
- documents etudiants

## Regle de reprise

Avant de toucher un domaine métier, vérifier si une migration récente a déjà formalisé le besoin. Une partie importante des comportements visibles dépend de ces migrations.



# Migration Firebase → Supabase

Guide pour migrer certaines données de Firebase RTDB vers Supabase Postgres.

## Principe

- Exporter les données RTDB (JSON) pour une ressource (ex: `Places`)
- Utiliser un script SQL d'import qui lit un blob JSON et insère dans les tables cibles
- Normaliser types/valeurs (booleans, nombres, dates)

## Import des places (existant)

Fichier: `supabase_migrations/20251106_import_places_from_json.sql`

Étapes:

1. Ouvrir le fichier et coller l'export JSON RTDB entre le bloc `$$ ... $$` (section `WITH src AS (...)`)
2. Exécuter avec `psql` sur la base Supabase

```
psql "$DATABASE_URL" -f
supabase_migrations/20251106_import_places_from_json.sql
```

Le script:

- Parse le JSON source
- Mappe les champs connus (spécialités, langues, PFPx, fileURL, etc.)
- Insère dans `public.places` (schéma défini par `20251028_create_places.sql`)

## Mappage courant (exemple)

- RTDB `IDPlace` → `PlaceId`
- RTDB `NomPlace` → `NomPlace`
- RTDB `fileURL` → `fileURL`
- RTDB `PFP1A`, `PFP2`, ... → colonnes JSONB `PFP1A`, `PFP2`, ...
- RTDB `MSQ`, `REHAB`, `AMBU`, `NEUROGER`, `SYSINT` → booleans de même nom
- RTDB `FR`, `DE`, `IT`, `ENG` → booleans langues
- `praticiensFormateurs` → `praticiensFormateurs` (`TEXT[]`)

## Conseils

- Sauvegarder la base avant tout import (dump)
- Itérer par petits lots, valider puis élargir
- Uniformiser les booléens ("true" → true), nombres ('1' → 1)
- Garder une trace des correspondances de champs dans ce fichier
- Ajouter des index après import si des colonnes deviennent critiques

## Vérifications

```
-- Total lignes importées
select count(*) from public.places;
-- Exemples de contrôles
select PlaceId, NomPlace from public.places where fileURL is null;
select * from public.places where "PFP1A"::text = '{} '::text;
```

# Routeur Multi-Systèmes

Cette page explique l'architecture "multi-systèmes" de l'app: Auth/RTDB Firebase et données/permissions Supabase.

## Composants clés

- `firebase.js`: init Firebase (Auth, RTDB, Storage). Variables `VITE_FIREBASE_*` requises.
- `src/supabase.js`: client Supabase (`VITE_SUPABASE_URL`, `VITE_SUPABASE_KEY`).
- `src/stores/role.js`: charge la session Supabase et les permissions via RPC, expose `can()` et `isSuper`.
- `src/router.js`: guard global `beforeEach` gère `requiresAuth`, `need`, `requiredRole` et charges dynamiques.

## Flux de navigation

1. Au premier passage: chargement des routes dynamiques (Supabase) via `addDynamicRoutesToRouter(router)`.
2. Vérification de l'état d'auth (Firebase ou Supabase) via `authStore.checkAuthState()`.
3. Initialisation des permissions (`roleStore.init()` → session Supabase + RPC permissions).
4. Application des règles de route:
  - `need: 'public' | 'anonymous'` → autorisé
  - `need` défini et pas d'utilisateur → redirect `'/'`
  - `need` défini → `isSuper` ou `roleStore.can(need)`
  - `requiresAuth` sans utilisateur → redirect `'/'`
  - `requiredRole` (héritage) → vérification via `rolesService`

## Exemple de métadonnées de route

```
{
  path: '/admin/routes-editor',
  component: DynamicRoutesEditorView,
  meta: { requiresAuth: true, need: ['super.all', 'admin'] }
}
```

## Permissions côté base

- Permissions calculées/storées dans `user_profiles.permissions[]` (voir migration `add_permissions_to_user_profiles.sql`).
- RPC d'exemple: `get_user_permissions(uid)` qui retourne `TEXT[]`.

## Recommandations

- Centraliser l'auth et les permissions dans des stores (Pinia)
- Éviter la logique d'autorisation dans les vues; utiliser `roleStore.can()`
- Documenter la taxonomie de permissions par domaines (`admin.*`, `page*.access`, `fp.*`)

# Backend Node et API serveur

## CÔTÉ SERVEUR

Le backend ne sert pas seulement d'appoint : certains flux sensibles ou multi-étapes passent réellement par lui.

API



### Routes serveur

Exposition d'endpoints métier et d'intégrations externes.

SEC



### Secrets & flux sensibles

Cas où le navigateur ne doit pas porter seul la logique.

OPS



### Workers & scripts

Imports, push, seed et automatisations techniques.

## Rôle du backend

Le serveur Node/Express dans `backend/` n'est pas accessoire. Il sert de couche applicative pour plusieurs besoins qui ne vivent pas proprement dans le frontend :

- routes API métier
- intégrations externes
- uploads FTP
- push
- certains flux reliés à Supabase

## Point d'entrée

- fichier principal : `backend/index.js`

- package dédié : `backend/package.json`

## Architecture visible

```
backend/  
|-- index.js  
|-- supabase/  
|   |-- institutionsStoreBackend.js  
|   |-- praticiensStoreBackend.js  
|   |-- postsBackendStore.js  
|   |-- communitiesStoreBackend.js  
|   |-- feedbackaBackend.js  
|   `-- ...  
|-- uploads/  
|   `-- ftpRoutes.js  
|-- workers/  
|   `-- pushOutboxWorker.js  
`-- scripts divers d'import, seed, migration et diagnostic
```

## Endpoints déclarés dans `backend/index.js`

Le serveur monte notamment :

- `/api/institutions`
- `/api/communities`
- `/api/enseignants`
- `/api/filePhysio`
- `/api/hashtags`
- `/api/posts`
- `/api/praticiens`
- `/api/resultat-votation`
- `/api/feedbacka`
- `/api/push`
- `/api/ftp`
- `/api/chat`

Et fournit aussi :

- `/api/ping`
- `/health`

# Quand passer par le backend plutôt que par le frontend

Utiliser le backend si :

- la logique a besoin de secrets serveur
- il faut contacter un service externe non exposé au navigateur
- le flux métier doit être centralisé côté serveur
- il faut encapsuler des opérations sensibles ou multi-étapes

Ne pas l'utiliser juste pour contourner un problème de modélisation front ou de policy RLS.

## Fichiers à auditer en priorité

- `backend/index.js`
- `backend/supabaseClient.js`
- `backend/uploads/ftpRoutes.js`
- `backend/workers/pushOutboxWorker.js`

# Catalogue détaillé des endpoints backend

## API SERVEUR

Cette page complète la cartographie des routes backend avec une lecture plus orientée exploitation et reprise.

## Point d'entrée serveur

Le montage des endpoints est fait dans `backend/index.js`.

## Endpoints métier montés

| Endpoint                            | Source montée                             | Usage                 |
|-------------------------------------|-------------------------------------------|-----------------------|
| <code>/api/institutions</code>      | <code>institutionsStoreBackend</code>     | institutions          |
| <code>/api/communities</code>       | <code>communitiesStoreBackend</code>      | communautés           |
| <code>/api/enseignants</code>       | <code>enseignantsStoreBackend</code>      | enseignants           |
| <code>/api/filePhysio</code>        | <code>filePhysioBackendStore</code>       | fichiers physio       |
| <code>/api/hashtags</code>          | <code>hashtagStoreBackend</code>          | hashtags              |
| <code>/api/posts</code>             | <code>postsBackendStore</code>            | feed social           |
| <code>/api/praticiens</code>        | <code>praticiensStoreBackend</code>       | praticiens            |
| <code>/api/resultat-votation</code> | <code>resultatVotationStoreBackend</code> | résultats de votation |
| <code>/api/feedbacka</code>         | <code>feedbackaBackend</code>             | outil Feedbacka       |
| <code>/api/push</code>              | <code>pushBackend</code>                  | notifications push    |



| Endpoint  | Source montée           | Usage                    |
|-----------|-------------------------|--------------------------|
| /api/ftp  | ftpRoutes               | uploads / diagnostic FTP |
| /api/chat | careconversStoreBackend | chat stateful            |

## Endpoints techniques et de test

| Endpoint                        | Rôle                    |
|---------------------------------|-------------------------|
| /api/ping                       | ping simple             |
| /health                         | healthcheck infra       |
| /api/ftp/diagnostic             | diagnostic direct FTP   |
| /api/praticiens_formateurs-test | route test              |
| /api/pong                       | test                    |
| /api/pongg                      | test                    |
| /api/chapters                   | exemple / test Supabase |

## Points importants de lecture

### CORS

Les origines autorisées diffèrent selon `NODE_ENV` :

- production : domaines HEdS explicitement listés ;
- développement : localhost et 127.0.0.1 sur ports connus.

### Parsing JSON

Le serveur accepte des payloads JSON jusqu'à `10mb`.

### Debug local

En environnement non production, un middleware loggue les requêtes entrantes.

# Ce qui est exploitable versus ce qui est historique

Le fait qu'un fichier existe dans `backend/supabase/` ne veut pas dire qu'il est réellement monté.

Toujours distinguer :

- les routes effectivement branchées dans `backend/index.js` ;
- les fichiers auxiliaires ;
- les scripts d'import, seed ou test ;
- les reliquats legacy.

## Réflexe de diagnostic backend

Quand un appel échoue :

1. vérifier que l'endpoint est bien monté dans `backend/index.js` ;
2. vérifier que le frontend appelle le bon chemin ;
3. vérifier les variables serveur ;
4. vérifier CORS ;
5. vérifier ensuite Supabase, FTP ou le service externe concerné.

# Traçabilité frontend ↔ backend ↔ données

## CHAÎNES TECHNIQUES

Cette page documente les flux les plus importants observés entre stores/services frontend, endpoints backend et données.

## Flux social

### Front

- `src/stores/postsStore.js`

### Appels

- `GET /api/posts`
- `POST /api/posts`
- `PUT /api/posts/:postId`
- `DELETE /api/posts/:postId`

### Backend

- `backend/supabase/postsBackendStore.js`

### Données associées

- `posts`
- `post_media`
- `hashtags`
- RPC `create_post_with_hashtags`

## Flux Feedbacka

### Front

- `src/stores/feedbackaStore.js`

## Appels

- `GET /api/feedbacka`
- `GET /api/feedbacka/:id`
- `POST /api/feedbacka`
- `PUT /api/feedbacka/:id`
- `POST /api/feedbacka/:id/test`
- `POST /api/feedbacka/:id/submit`
- `GET /api/feedbacka/:id/submissions`

## Backend

- `backend/supabase/feedbackaBackend.js`

## Flux résultats de votation

### Front

- `src/service/resultatVotationService.js`

## Appels backend observés

- `POST /api/resultat-votation/run-algorithm`
- `POST /api/resultat-votation/confirm-algorithm`
- `GET /api/resultat-votation/assignment-counts/:pfpType/:year`
- `GET /api/resultat-votation/pfp3-proposals/:year`
- `POST /api/resultat-votation/save-pfp3-proposals`
- `POST /api/resultat-votation/generate-pfp4-proposals`
- `POST /api/resultat-votation/save-pfp4-proposals`
- `GET /api/resultat-votation/pfp4-proposals/:year`
- `GET /api/resultat-votation/results/:pfpType/:year`
- `GET /api/resultat-votation/student/:userId/:pfpType/:year`
- `GET /api/resultat-votation/statistics/:pfpType/:year`
- `PUT /api/resultat-votation/status/:resultId`
- `DELETE /api/resultat-votation/:resultId`
- `DELETE /api/resultat-votation/algorithm-run/:algorithmRunId`

## Backend

- `backend/supabase/resultatVotationStoreBackend.js`

## Données associées

- `student_result_vote`
- `vote_place_aggregation`
- RPC `get_algorithm_results`
- RPC `get_student_result`

## Flux institutions / médias

### Front

- `src/service/institutionMediaService.js`

### Appels

- `POST /api/institutions/:institutionId/images`
- `DELETE /api/institutions/:institutionId/images`

### Backend

- `backend/supabase/institutionsStoreBackend.js`

## Données associées

- table `institutions`
- bucket storage `institutions`

## Flux FTP

### Front

- `src/views/apps/tools/FTPUploadTestView.vue`

### Appels

- `POST /api/ftp/upload`
- `GET /api/ftp/diagnostic`

### Backend

- `backend/uploads/ftpRoutes.js`

## Flux rôles et permissions

## Front

- `src/stores/role.js`
- `src/views/admin/users/ManageUserRoles.vue`
- `src/views/admin/security/RBACAdminView.vue`

## Accès observés

- RPC `api_my_permissions`
- RPC `update_user_permissions`
- RPC `get_user_permissions`
- lecture/écriture directe :
  - `permissions`
  - `roles`
  - `role_permissions`
  - `user_roles`
  - `user_profiles`

## Principe de diagnostic

Quand un flux casse, identifier :

1. le point d'entrée frontend ;
2. le store ou service appelant ;
3. l'endpoint ou la table réellement utilisée ;
4. le fichier backend qui porte la logique ;
5. la table ou la RPC finale.

# Cartographie des routes backend

## Objectif

Avoir une lecture rapide de ce que sert le backend Express et ou se trouve chaque route.

## Point d'entree

- declaration globale: `backend/index.js`

## Routes montees

| Prefixe                             | Fichier principal                                             | Domaine         |
|-------------------------------------|---------------------------------------------------------------|-----------------|
| <code>/api/institutions</code>      | <code>backend/supabase/institutionsStoreBackend.js</code>     | institutions    |
| <code>/api/communities</code>       | <code>backend/supabase/communitiesStoreBackend.js</code>      | communities     |
| <code>/api/enseignants</code>       | <code>backend/supabase/enseignantsStoreBackend.js</code>      | enseignants     |
| <code>/api/filePhysio</code>        | <code>backend/supabase/filePhysioBackendStore.js</code>       | fichiers physio |
| <code>/api/hashtags</code>          | <code>backend/supabase/hashtagStoreBackend.js</code>          | hashtags        |
| <code>/api/posts</code>             | <code>backend/supabase/postsBackendStore.js</code>            | social          |
| <code>/api/praticiens</code>        | <code>backend/supabase/praticiensStoreBackend.js</code>       | praticiens      |
| <code>/api/resultat-votation</code> | <code>backend/supabase/resultatVotationStoreBackend.js</code> | votation        |
| <code>/api/feedbacka</code>         | <code>backend/supabase/feedbackaBackend.js</code>             | feedback        |
| <code>/api/push</code>              | <code>backend/supabase/pushBackend.js</code>                  | push            |
| <code>/api/ftp</code>               | <code>backend/uploads/ftpRoutes.js</code>                     | FTP             |
| <code>/api/chat</code>              | <code>backend/supabase/careconversStoreBackend.js</code>      | chat state      |

## Routes techniques directes

| Route                            | Usage                             |
|----------------------------------|-----------------------------------|
| <code>/health</code>             | healthcheck                       |
| <code>/api/ping</code>           | test simple                       |
| <code>/api/ftp/diagnostic</code> | verification config FTP           |
| <code>/api/chapters</code>       | test Supabase / exemple technique |

## Exemple concret: `postsBackendStore.js`

Cette route expose un CRUD posts simplifié:

- `POST /api/posts/`
- `GET /api/posts/`
- `PUT /api/posts/:id`
- `DELETE /api/posts/:id`

Particularités:

- creation via RPC `create_post_with_hashtags`
- enrichissement avec `author:user_profiles`, hashtags, likes, replies
- la suppression s'appuie sur les permissions de la base

## Ce qu'il faut verifier avant de modifier une route backend

1. le frontend consomme-t-il directement cette route ?
2. existe-t-il déjà un service front ou un store qui dépend de sa forme de réponse ?
3. une RPC ou une policy RLS est-elle impliquée ?
4. la route utilise-t-elle un client standard ou un client admin ?

## Clients Supabase cote backend

Le backend dispose de deux modes dans `backend/supabaseClient.js`:

- client normal, avec RLS



- client admin si `SUPABASE_SERVICE_ROLE_KEY` est disponible

Implication:

- certaines operations peuvent contourner RLS
- il faut documenter explicitement les cas ou ce bypass est assume

## **Dette visible**

Le dossier `backend/supabase/` contient aussi des copies, scripts d'import et fichiers non finalises. Ils ne doivent pas etre confondus avec les routes actives.

# Backend actif versus fichiers legacy

## Pourquoi cette page

Le dossier `backend/` contient a la fois:

- des routes actives
- des scripts d'import
- des fichiers de test
- des copies
- des fichiers provisoires

Sans distinction claire, il est facile de modifier le mauvais fichier.

## Fichiers backend actifs en production applicative

### Serveur et configuration

- `backend/index.js`
- `backend/supabaseClient.js`

### Routes montees

- `backend/supabase/careconversStoreBackend.js`
- `backend/supabase/communitiesStoreBackend.js`
- `backend/supabase/enseignantsStoreBackend.js`
- `backend/supabase/feedbackaBackend.js`
- `backend/supabase/filePhysioBackendStore.js`
- `backend/supabase/hashtagStoreBackend.js`
- `backend/supabase/institutionsStoreBackend.js`
- `backend/supabase/postsBackendStore.js`
- `backend/supabase/praticiensStoreBackend.js`
- `backend/supabase/pushBackend.js`
- `backend/supabase/resultatVotationStoreBackend.js`
- `backend/uploads/ftpRoutes.js`

### Worker actif potentiel

- `backend/workers/pushOutboxWorker.js`

# Fichiers non prioritairement applicatifs

## Imports / seeds / migration

- `importUsers.js`
- `importPosts.js`
- `importPlaces.js`
- `importInstitutions.js`
- `importEnseignants.js`
- `importCommunities.js`
- `importPraticiensFormateurs.js`
- `exportPlacesFromFirebase.js`
- scripts `seed_*`, `restore-*`, `migrate-*`

## Tests et diagnostics

- `test-algo-v4.js`
- `test-algo-v4-sim.js`
- `parse-votes.js`
- `export-pfp4-criteres.js`

## Fichiers a traiter comme dette

- `careconversStoreBackend copy.js`
- `careconversStoreBackend copy 2.js`
- `Untitled-1.js`
- `todo`

## Regle de travail

Quand tu modifies `backend/`:

1. commencer par verifier si le fichier est effectivement monte dans `backend/index.js`
2. ignorer les copies sauf si elles servent explicitement de référence historique
3. considerer les fichiers d'import comme outillage, pas comme logique applicative vivante

## Recommandation

A terme, il faudrait isoler:

- `backend/routes/`
- `backend/scripts/`
- `backend/legacy/`

Aujourd'hui ce n'est pas le cas, donc la lecture de `backend/index.js` reste la source de vérité.

# Firestore - vue d'ensemble

## Rôle dans le projet

Firestore reste présent comme socle legacy sur plusieurs flux:

- authentication
- Realtime Database
- Storage
- Hosting

## Fichier principal

- `src/firebase.js`

## Variables attendues

```
VITE_FIREBASE_API_KEY=  
VITE_FIREBASE_AUTH_DOMAIN=  
VITE_FIREBASE_PROJECT_ID=  
VITE_FIREBASE_STORAGE_BUCKET=  
VITE_FIREBASE_MESSAGING_SENDER_ID=  
VITE_FIREBASE_APP_ID=  
VITE_FIREBASE_DATABASE_URL=
```

## Comportement important

Si ces variables ne sont pas présentes, le frontend désactive Firestore et loggue les clés manquantes.

## À retenir

Même si Supabase devient la cible principale, Firestore ne doit pas être traité comme mort tant que les flux legacy existent encore.

# Firestore Realtime Database

## Rôle

La Realtime Database porte encore une partie des données historiques du projet.

## Fichier principal

- `src/firebase.js`

## Usage typique

- données legacy utilisateurs
- rôles historiques
- structures encore non migrées

## Réalité de maintenance

Avant de modifier un flux supposé legacy, vérifier s'il lit encore vraiment la RTDB ou s'il a déjà été déplacé vers Supabase.

# Firestore Storage

## Rôle

Firestore Storage reste utilisé pour une partie des fichiers.

## Point d'attention

Le stockage n'est pas uniformisé sur tout le projet. Certains flux sont pensés pour migrer vers Supabase Storage.

## Conséquence

Quand un problème de document ou média apparaît, il faut d'abord identifier quel bucket ou quel service est réellement utilisé.

# Firestore Auth

## Rôle

Firestore Auth reste utilisé pour une partie des sessions legacy.

## Points de référence

- `src/firebase.js`
- `src/stores/authStore.js`

## Intégration

`authStore` expose:

- `signUpFirestore`
- `signInFirestore`
- `resetPasswordFirestore`

et écoute aussi `onAuthStateChanged`.

## Point de vigilance

Le projet étant hybride, une correction auth doit toujours être vérifiée aussi côté Supabase.



# Supabase - vue d'ensemble

## Rôle dans le projet

Supabase est la cible principale pour:

- l'auth moderne
- la base relationnelle
- les politiques RLS
- le stockage moderne

## Fichier principal côté frontend

- `src/supabase.js`

## Comportements importants

- lecture de `VITE_SUPABASE_URL`
- lecture de `VITE_SUPABASE_KEY`
- normalisation de l'URL si `/rest/v1` est fourni par erreur
- configuration de session persistante avec PKCE

## Conséquence de maintenance

Une évolution Supabase peut impacter:

- auth
- rôles et permissions
- stores Pinia
- services métier
- backend Express

# Services Supabase

## Principe

Les accès Supabase doivent rester centralisés dans les services métier ou dans des stores bien identifiés.

## Niveaux concernés

- `src/service/`
- `src/stores/`
- `backend/supabase/` côté serveur

## Bonnes pratiques

- éviter les requêtes brutes dispersées dans les vues
- laisser les vues orchestrer, pas porter la logique de données
- penser RLS et permissions au moment de la requête, pas après

## Exemples de services critiques

- `rolesService.js`
- `studentsService.js`
- `planningService.js`
- `resultatVotationService.js`
- `gamificationServiceSupabase.js`

# Supabase RLS et sécurité

## Rôle

RLS sert de dernier niveau d'enforcement côté base.

## Ce qu'il faut garder en tête

- un accès autorisé par le front peut rester refusé par la base
- un compte admin front ne doit pas faire oublier la sécurité base
- le backend peut parfois utiliser une service key et contourner RLS

## À vérifier pour chaque table sensible

- `select`
- `insert`
- `update`
- `delete`

## Zones critiques

- `user_profiles`
- tables PFP
- tables de votation
- tables de stockage et documents

# Migrations Supabase

## Source de vérité

Les migrations réellement présentes sont dans:

- `supabase/migrations/`

## Ce qu'il faut faire

- appliquer les migrations dans l'ordre logique
- vérifier l'impact métier avant exécution
- relire les changements liés aux permissions et au stockage

## Migrations structurantes

- permissions via `api_my_permissions`
- planning et classes
- votation et priorités
- storage institutions, avatars, documents étudiants

## Référence utile

Voir aussi:

- `data/migrations-catalog`
- `data/schema-supabase`

# Domaine métier - formation pratique

## Résumé

La formation pratique est un des plus gros blocs métier du projet. Elle couvre :

- gestion des places
- affectations
- praticiens
- cohortes et étudiants
- workflows administratifs liés à la PFP

## Risques principaux

- droits historiques parfois portés par `page1.access`
- forte dépendance à la qualité des données
- coexistence entre logique frontend, services et scripts

## Réflexe de diagnostic

Quand un flux PFP ne fonctionne pas :

1. vérifier la permission
2. vérifier le store et le service appelés
3. vérifier la source de données réelle
4. vérifier si un script ou une migration prépare les données

# Traçabilité détaillée de la formation pratique

## PFP DÉTAILLÉE

Cette page sert de carte de reprise détaillée pour le domaine le plus dense du projet.

## Principe

La formation pratique ne se comprend pas écran par écran. Il faut relier :

- les vues ;
- les stores ;
- les services ;
- le backend ;
- les tables Supabase ;
- les scripts ou migrations qui préparent parfois la donnée.

## Matrice ciblée PFP

| Fonction              | Vue / zone                                   | Store / service                            | Back                 |
|-----------------------|----------------------------------------------|--------------------------------------------|----------------------|
| Gestion des places    | admin/pfp/ManagementPlaces*, PlacesViewPHYFP | placesStore + services PFP                 | parfois Supabase     |
| Institutions PFP      | InstitutionsViewPHYFP, vues secrétariat      | services métier                            | direct               |
| Praticiens formateurs | PraticiensFormateurViewPHYFP                 | praticiensStore, praticiensFormateursStore | /api/ ou directement |
| Résultats de votation | VotationGenericView, previews, résultats     | resultatVotationService                    | /api/ votation       |

| Fonction              | Vue / zone                                       | Store / service                                                          | Back   |
|-----------------------|--------------------------------------------------|--------------------------------------------------------------------------|--------|
| Vote étudiant         | vues de votation                                 | <code>votesBackendService</code>                                         | RPC S  |
| Cohortes et étudiants | <code>EtudiantsViewPHYFP</code> ,<br>secrétariat | <code>studentsService</code>                                             | direct |
| Cas particuliers      | <code>SuiviCasParticuliers.vue</code>            | logique vue + données directes                                           | direct |
| Alertes PFP           | <code>AlertesDashboard.vue</code>                | <code>pfpAlertsService</code> ,<br><code>intelligentAlertsService</code> | mixte  |

## Tables les plus sensibles

Les tables qui reviennent le plus dans le domaine sont :

- `StudentsPhysio`
- `places`
- `institutions`
- `praticiens_formateurs`
- `student_result_vote`
- `votation_sessions`
- `suivi_cas_particuliers`
- `recap_cpt_evaluation`
- `user_profiles`

## Vues PFP très critiques

En reprise technique, surveiller particulièrement :

- `src/views/admin/pfp/PlacesAssignedView.vue`
- `src/views/admin/pfp/ManagementVotationPrioritaireView.vue`
- `src/views/admin/pfp/ManagementOffreView.vue`
- `src/views/admin/votations/VotationGenericView.vue`
- `src/views/admin/formation-pratique/secretariat/VueDEnsembleFP.vue`

- `src/views/admin/formation-pratique/secretariat/VerificationCriteresEtudiants.vue`

## Lecture recommandée d'un écran PFP

Pour un écran PFP important :

1. identifier le route module (`pfp.js` ou `votations.js`) ;
2. identifier la vue ;
3. relever les appels Supabase directs ;
4. relever les services appelés ;
5. relever les endpoints `/api/resultat-votation` si présents ;
6. vérifier les tables manipulées ;
7. vérifier la permission `page1.access` ou équivalent.

## Signal d'architecture à retenir

La PFP est un domaine hybride :

- une partie passe par services backend ;
- une partie lit directement Supabase ;
- une partie dépend de conventions legacy et de données historiques ;
- une partie du métier est encodée dans les notes, flags et structures JSON.

## Réflexe avant modification

Avant de changer un flux PFP :

- identifier l'impact sur `student_result_vote` ;
- vérifier si `votation_sessions` stocke des propositions liées ;
- vérifier si la vue dépend d'un résultat déjà publié ;
- vérifier si les places déjà validées doivent être exclues ;
- vérifier si des critères manquants ou cas particuliers entrent dans le calcul.



# Services PFP, étudiants et votes

## SERVICES MÉTIER

resultatVotationService

studentsService

votesBackendService

Cette page documente les services les plus sensibles du domaine formation pratique et votation.

### studentsService.js

Rôle observé :

- servir de couche de récupération et de fusion des données étudiantes ;
- privilégier `user_profiles` comme source plus fiable pour les identités ;
- enrichir ensuite depuis `StudentsPhysio`.

Points importants :

- le service assume une coexistence de sources ;
- il essaie plusieurs sélections selon les colonnes réellement disponibles ;
- il maintient un cache simple module-level ;
- il mappe la classe via `StudentsPhysio` quand nécessaire.

Signal d'architecture :

- les étudiants ne vivent pas dans une seule table parfaitement propre ;
- la reprise doit toujours vérifier la cohérence `user_profiles` ↔ `StudentsPhysio`.

### votesBackendService.js

Rôle observé :

- encapsuler les RPC de vote ;
- centraliser les opérations de lecture/écriture de vote étudiant ;
- éviter d'écrire directement dans la table depuis les écrans.

RPC utilisées :

- `get_student_vote`
- `upsert_student_vote`
- `delete_student_vote`
- `has_student_voted`
- `get_all_student_votes`
- `count_votes`
- `get_top_voted_places`

Lecture complémentaire :

- `vote_statistics`
- `vote_place_aggregation`

Signal d'architecture :

- le vote étudiant est fortement orienté RPC ;
- la logique métier de vote ne vit pas uniquement dans les vues.

## **resultatVotationService.js**

Rôle observé :

- piloter les endpoints backend de calcul et de persistance des résultats ;
- servir de façade front pour l'algorithme de votation ;
- mixer endpoints backend et RPC/lectures Supabase directes selon le besoin.

Endpoints principaux :

- run / confirm algorithm ;
- résultats ;
- étudiant ;
- statistiques ;
- status ;
- suppression de résultat ;
- suppression d'exécution ;
- génération et sauvegarde PFP3/PFP4.

Points importants :

- le service gère explicitement les tokens Supabase ;

- une partie des lectures simples passe directement par Supabase ;
- les calculs lourds passent par Express.

## Réflexe de reprise

Quand un problème touche la PFP :

1. vérifier la source étudiant dans `studentsService` ;
2. vérifier si le vote passe par RPC ou backend HTTP ;
3. vérifier si le résultat final est lu depuis `student_result_vote` ;
4. vérifier les propositions `votation_sessions` si PFP3/PFP4.

# Flux fonctionnels de la formation pratique

## Objectif

Documenter les grands flux PFP pour pouvoir reprendre le domaine sans se limiter a une liste de fichiers.

## Flux 1 - gestion structurelle des referentiels

Ce flux concerne:

- institutions
- places
- praticiens formateurs
- etudiants

Points d'entree typiques:

- vues `admin/formation-pratique/*`
- stores `placesStore`, `praticiensStore`, `repondantPhysioHESStore`

Finalité :

- maintenir le referentiel utilise par les campagnes PFP

## Flux 2 - preparation des offres

Ce flux concerne:

- offres de places
- suivi institutions
- recapitulatifs
- secretariat

Vues typiques:

- `ManagementOffreView.vue`
- `SuiviInstitutions.vue`
- `SuiviOffresInstitutions.vue`

- `TableauRecapitulatifOffres.vue`

Risque:

- toucher aux offres sans verifier les consequences sur la votation et les affectations

## Flux 3 - votation prioritaire

Ce flux concerne:

- collecte ou lecture des priorites
- vues de resultat
- cas specifiques ou particuliers

Vues typiques:

- `ManagementVotationPrioritaireView.vue`
- `VotationPrioritaireViewPHYFP.vue`
- `ResultPreviewVotationView.vue`

## Flux 4 - votation PFP

Ce flux concerne:

- campagne principale
- etudiants
- resultats
- preview

Vues typiques:

- `VotationEtudiantsView.vue`
- `VotationPFPViewPHYFP.vue`
- `ResultPreviewVotationView.vue`

## Flux 5 - affectation et validation

Ce flux concerne:

- places assignees

- validation des places
- validation PFP
- cohort stats

Vues typiques:

- `PlacesAssignedView.vue`
- `PlacesAssignmentView.vue`
- `ValidationPFP.vue`
- `PfpCohortStatsView.vue`

## Flux 6 - secretariat et supervision

Ce flux concerne:

- verification des criteres
- suivi cas particuliers
- alertes
- vues d'ensemble
- recap notes / CPT

Vues typiques:

- `VerificationCriteresEtudiants.vue`
- `SuiviCasParticuliers.vue`
- `AlertesDashboard.vue`
- `VueDEnsembleFP.vue`
- `RecapPFPNotes.vue`
- `RecapCPTEvaluation.vue`

## Schema simplifie

## Regle de maintenance

Ne jamais traiter un ecran PFP comme isole. Il faut toujours se demander a quel flux il appartient, ce qui vient avant et ce que cela impacte apres.

# Domaine métier - planning et soins infirmiers

## Périmètre

Ce domaine regroupe :

- années académiques
- planning journalier
- planning hebdomadaire
- planning semestriel
- planning annuel
- workload et feuilles de charge
- dashboards et workflows soins infirmiers

## Emplacements principaux

### Vues

- `src/views/admin/planning/`
- `src/views/planning/`
- `src/views/admin/soins-infirmiers/`

### Services et composables

- `src/service/planningService.js`
- `src/service/academicPlanningService.js`
- `src/service/workloadService.js`
- `src/service/academicYearService.js`
- `src/composables/useAcademicYear.js`
- `src/composables/useModulePlanning.js`

## Données sensibles pour ce domaine

- année académique active
- périodes et semestres
- classes
- modules
- cours

- périodes asynchrones

## **Nature du risque**

Un bug dans ce domaine crée souvent :

- des incohérences de calendrier
- des affectations erronées
- des dashboards trompeurs
- des vues vides si l'année académique n'est pas correctement résolue

## **Règle de diagnostic**

Quand une vue planning est vide ou incohérente :

1. vérifier l'année académique sélectionnée
2. vérifier la structure de données module/cours/période
3. vérifier si la vue dépend d'un store local ou d'un service admin
4. vérifier les migrations récentes touchant les périodes



# Domaine métier - social, engagement et gamification

## DOMAINE TRANSVERSE

Ce domaine ne vit pas en silo : il dépend des profils, des interactions, des services backend et de la logique d'engagement.

SOC



### Bloc social

Feed, hashtags, mentions, communautés et interactions utilisateurs.

GAM



### Gamification

XP, niveaux, quêtes, défis, badges et maisons HES.

RISK



### Effets de bord

Les événements sociaux et auth peuvent impacter le scoring et les récompenses.

## Bloc social

Le bloc social couvre :

- feed
- hashtags
- mentions
- communautés
- contenu partagé
- interactions entre profils

## Emplacements principaux

- `src/views/social/`
- `src/stores/postsStore.js`
- `src/stores/chatStore.js`
- `backend/supabase/postsBackendStore.js`
- `backend/supabase/communitiesStoreBackend.js`
- `backend/supabase/hashtagStoreBackend.js`

## Bloc gamification

La gamification couvre :

- maisons HES
- XP
- niveaux
- quêtes
- défis
- badges
- analytics

## Emplacements principaux

- `src/service/gamificationServiceSupabase.js`
- `src/service/gamificationIntegration.js`
- `src/service/challengesService.js`
- `src/service/questsService.js`
- `src/service/badgesService.js`
- `src/views/admin/gamification/`

## Caractéristique importante

Ce domaine est transverse. Une action banale, comme un login ou une complétion d'action, peut impacter la gamification.

## Risques fréquents

- sous-estimer les effets de bord d'une modification auth
- manipuler l'XP dans la vue au lieu du service
- modifier le social sans vérifier les flux backend correspondants

# Gamification Supabase et services d'engagement

## ENGAGEMENT

Cette page documente le service Supabase de gamification, qui sert de base à l'affichage des maisons et d'une partie des stats utilisateur.

### `gamificationServiceSupabase.js`

Rôle observé :

- lire les données de gamification utilisateur ;
- reconstituer les informations maison ;
- calculer le niveau et l'XP vers le niveau suivant ;
- mettre en cache localement certaines réponses.

## Tables observées

- `gamification_data`
- `houses`

## Ce que le service recompose

Pour un utilisateur, le service expose un objet enrichi :

- maison ;
- niveau ;
- XP totale ;
- XP restante ;
- house info ;
- badges / quêtes / challenges simplifiés côté lecture.

## Logique notable

- cache local 5 minutes ;
- calcul du niveau dérivé depuis l'XP ;
- mapping maison par `house_id` ;
- fallback propre si aucune donnée n'existe.

## Domaine plus large

Le service ne couvre pas tout le domaine engagement.

Autres services à connaître :

- `gamificationIntegration.js`
- `gamificationService.js`
- `badgesService.js`
- `questsService.js`
- `challengesService.js`
- `hesHousesService.js`
- `adminQuestsService.js`

## Risques de maintenance

- modifier la structure d'XP sans aligner le calcul de niveau ;
- modifier `houses` sans vérifier les vues de profil ;
- oublier que le cache peut masquer temporairement un changement.

# Algorithme de votation et d'attribution

## NOYAU MÉTIER

Cette page synthétise le comportement réellement observé dans `backend/supabase/resultatVotationStoreBackend.js`.

## Point d'entrée technique

Le cœur de l'algorithme vit dans :

- `backend/supabase/resultatVotationStoreBackend.js`

Endpoints clés :

- `POST /api/resultat-votation/run-algorithm`
- `POST /api/resultat-votation/confirm-algorithm`
- `GET /api/resultat-votation/results/:pfpType/:year`
- `GET /api/resultat-votation/student/:userId/:pfpType/:year`
- `GET /api/resultat-votation/statistics/:pfpType/:year`

Pour les propositions spécifiques :

- `GET /api/resultat-votation/pfp3-proposals/:year`
- `POST /api/resultat-votation/save-pfp3-proposals`
- `POST /api/resultat-votation/generate-pfp4-proposals`
- `POST /api/resultat-votation/save-pfp4-proposals`
- `GET /api/resultat-votation/pfp4-proposals/:year`

## Ce que l'algorithme fait d'abord

Avant toute attribution, le backend :

1. vérifie l'authentification ;
2. vérifie l'accès admin ;
3. charge les assignations existantes ;

4. calcule la capacité déjà consommée ;
5. exclut les étudiants déjà assignés si nécessaire.

Conséquence importante :

- l'algorithme n'écrase pas naïvement l'existant ;
- il peut conserver des assignations manuelles, prioritaires ou déjà publiées.

## Données prises en compte

Les calculs s'appuient notamment sur :

- les étudiants à traiter ;
- les places disponibles ;
- la capacité de chaque place ;
- les critères couverts par chaque place ;
- les critères manquants de chaque étudiant ;
- le `priorityScore` ;
- les stages déjà réalisés ;
- les résultats déjà enregistrés.

## Étape 0 — Conservation de l'existant

Le backend charge :

- `student_result_vote`
- `StudentsPhysio`

Objectifs :

- repérer les places déjà assignées ;
- déduire la capacité restante ;
- ne pas reposer une place déjà effectuée ;
- réinjecter les assignations existantes dans le résultat final.

## Étape 1 — Construction du pool des places

Chaque place est transformée en structure exploitable :

- capacité initiale ;

- capacité restante ;
- nombre de votes ;
- étudiants déjà assignés ;
- critères couverts.

Critères métiers observés :

- MSQ
- SYSINT
- NEUROGER
- AIGU
- REHAB
- AMBU
- FR
- DE

## Étape 2 — Tri des étudiants

Les étudiants éligibles sont triés principalement par :

1. priorityScore décroissant ;
2. nom/prénom en second niveau.

Cela signifie que l'ordre de passage dans l'algorithme est déjà une décision métier.

## Étape 3 — Construction des places candidates

Pour chaque étudiant, le backend construit une liste de places candidates en tenant compte :

- des places déjà faites ;
- de la capacité restante ;
- des critères manquants de l'étudiant ;
- de certains cas particuliers comme l'allemand (DE) ;
- du nombre minimal de places proposées dans certains cas.

## Logique générique d'attribution

Le code documente une logique dite "votation générique".

Principe observé :

- les choix sont traités comme un pool de possibilités ;
- le but n'est pas seulement de respecter un rang ;
- l'objectif est d'attribuer la meilleure place disponible selon les critères manquants.

Tri des places candidates observé :

1. nombre de critères couverts ;
2. priorité à certains cas comme DE ;
3. popularité / pression sur la place ;
4. capacité restante ;
5. nom de place.

## Cas de fallback

Quand aucune place ne couvre réellement les critères manquants :

- l'étudiant peut quand même être assigné ;
- l'affectation est marquée comme fallback ;
- `assigned_rank = 99` sert de signal technique ;
- la note indique explicitement un cas "hors ciblage".

Point important :

- un résultat peut donc être valide techniquement, tout en étant sous-optimal métier.

## Résultat persisté

Les résultats sont upsertés dans :

- `student_result_vote`

Clé de conflit observée :

- `user_id, pfp_type, year`

Champs importants enregistrés :



- `assigned_place_id`
- `assigned_place_name`
- `assigned_institution_name`
- `assigned_rank`
- `algorithm_run_id`
- `original_choices`
- `priority_score`
- `notes`
- `status`
- `assigned_at`

## Confirm algorithm

L'endpoint `confirm-algorithm` sert à persister un aperçu déjà calculé.

Usage typique :

1. calcul d'un aperçu ;
2. revue ;
3. confirmation ;
4. upsert en base.

## PFP3 — Propositions

Pour PFP3, le backend sait :

- lire les propositions sauvegardées ;
- sauvegarder des propositions par étudiant ;
- stocker ces propositions dans `votation_sessions`.

Le payload stocké contient notamment :

- `placeIds`
- `missingCriteria`
- `appliedRule`
- éventuellement `_assignCounts`

## PFP4 — Génération de propositions

Pour PFP4, le backend applique des règles métier supplémentaires selon les critères manquants.

Règles observées dans le code :

- cas `DE` seul ;
- cas `DE + SYSINT` ;
- cas `SYSINT` seul ;
- cas `SYSINT + autres` ;
- cas autres critères sans `DE` ;
- cas sans critère manquant.

Le backend :

- calcule les propositions ;
- déduplique les places ;
- exclut les places déjà réalisées ;
- annote la règle appliquée ;
- sauvegarde ensuite les propositions dans `votation_sessions`.

## Statistiques produites

Le backend calcule aussi des stats utiles :

- nombre total d'étudiants ;
- nombre d'assignments conservées ;
- nombre d'assignments générées ;
- nombre d'échecs ;
- nombre de places utilisées ;
- nombre d'assignments fallback ;
- moyenne de critères couverts ;
- nombre d'étudiants avec zéro critère couvert.

## Pièges de maintenance

- croire que l'algorithme se limite aux préférences utilisateur ;
- oublier les assignments existantes déjà persistées ;
- oublier que la capacité est diminuée par l'historique conservé ;
- ignorer les critères manquants et les cas `DE` / `SYSINT` ;

- modifier le front sans vérifier les notes, statuts et conventions backend.

## Réflexe de diagnostic

Quand un résultat semble faux :

1. vérifier l'entrée étudiante ;
2. vérifier les places candidates ;
3. vérifier les critères manquants ;
4. vérifier les capacités déjà consommées ;
5. vérifier si le cas est un fallback ;
6. vérifier enfin ce qui a été persisté dans `student_result_vote`.

# Cartographie UI et ownership fonctionnel

VUE FRONTEND

src/views/

Cette page sert à retrouver rapidement où vit un domaine fonctionnel dans l'interface et dans l'admin.

## Arborescence de premier niveau

Répertoires principaux observés dans `src/views/` :

- `admin`
- `apps`
- `associations`
- `auth`
- `capsulesInsuffiance`
- `documentation`
- `home`
- `institutions`
- `media`
- `pages`
- `planning`
- `social`
- `users`

## Sous-domaines admin

Répertoires observés dans `src/views/admin/` :

- `academic`
- `courses`
- `formation-pratique`
- `gamification`
- `institutions`
- `lists`

- modules
- pfp
- places
- planning
- security
- soins-infirmiers
- tools
- users
- validation
- votations

## Lecture pratique

Quand tu cherches une fonctionnalité :

1. repère d'abord la vue ;
2. cherche ensuite le store associé ;
3. cherche ensuite le service ;
4. vérifie enfin si le backend ou Supabase portent la logique réelle.

## Domaines fortement administrés

Les zones les plus denses côté admin sont en général :

- formation pratique / PFP
- planning
- soins infirmiers
- sécurité / utilisateurs
- votations
- gamification

## Répartition indicative

| Zone UI              | Ce qu'on y trouve généralement            |
|----------------------|-------------------------------------------|
| views/admin/academic | pilotage académique, dashboards, KPI      |
| views/admin/planning | planification, workload, vues calendaires |

| Zone UI                        | Ce qu'on y trouve généralement              |
|--------------------------------|---------------------------------------------|
| views/admin/formation-pratique | administration large du domaine PFP         |
| views/admin/pfp                | campagnes, places, affectations, validation |
| views/admin/votations          | résultats, previews, gestion des sessions   |
| views/admin/security           | contrôle d'accès, sécurité, outils admin    |
| views/social                   | feed, communautés, interactions             |
| views/media                    | bibliothèque, vidéos, contenus              |
| views/apps                     | applications intégrées                      |

## Indices de couplage fort

Une vue mérite une attention particulière si :

- elle dépend de plusieurs stores ;
- elle appelle directement plusieurs services ;
- elle dépend d'un prétraitement de données externe ;
- elle combine Firebase et Supabase ;
- elle applique à la fois logique métier et contrôle d'accès.

## Réflexe de reprise

Pour documenter ou modifier un écran important, toujours noter :

- son répertoire de vue ;
- ses stores ;
- ses services ;
- ses permissions ;
- sa source de données ;
- son impact sur les autres flux.

# Applications intégrées

## Périmètre

Le projet embarque plusieurs applications ou sous-modules intégrés :

- chat
- mail
- notes
- calendrier
- fichiers
- événements

## Logique générale

Ces applications partagent :

- le même socle frontend
- la même authentification
- les mêmes conventions UI
- une partie des services transverses

## Point d'attention

Avant de modifier une application intégrée, vérifier :

- son service principal dans `src/service/`
- son store éventuel
- sa source de données réelle
- ses dépendances UI ou permissions

# Modules et medias

## Perimetre

Le bloc media couvre:

- bibliotheque video
- modules video
- validation de contenus media
- pages de test Vimeo

## Emplacements principaux

- `src/views/media/`
- `src/views/admin/academic/VideoLibraryView.vue`
- `src/service/videoLibraryService.js`
- `src/service/vimeoService.js`
- `src/service/mediaService.js`



# Configuration Vimeo

## Variable attendue

```
VITE_VIMEO_ACCESS_TOKEN=
```

## Usage

Cette variable est utilisee pour les integrations video cote frontend et certains services lies a la bibliotheque video.

## Vigilance

- verifier la validite du token en environnement cible
- ne pas hardcoder de token dans le code
- tester les pages media et admin apres toute modification

# Services media

## Services a connaitre

- `src/service/mediaService.js`
- `src/service/vimeoService.js`
- `src/service/videoLibraryService.js`
- `src/service/institutionMediaService.js`

## Rôle

Ces services centralisent la logique de consultation, validation et organisation des contenus media.

## Regle

Éviter de brancher des appels Vimeo directement dans les vues. La logique d'accès doit rester dans les services.

# Exploitation locale et développement

## Commandes

```
npm run dev  
npm --prefix backend run dev  
npm run docs:dev
```

# Build, packaging et déploiement

## Surfaces à déployer

Le projet peut impliquer jusqu'à trois surfaces :

- frontend principal
- documentation Docusaurus
- backend Node/Express

## Build frontend

```
npm run build
```

Sortie :

- `dist/`

## Build documentation

```
npm run docs:build
```

Sortie :

- `documentation/build/`

## Build global

```
npm run build:all
```

Ce script :

1. build le frontend
2. build la documentation
3. copie les docs dans `dist/docs`

## Fichiers de déploiement à connaître

- `firebase.json`
- `docker-compose.dev.yml`
- `docker-compose.prod.yml`
- `Dockerfile.dev`
- `Dockerfile.frontend.prod`
- `deploy/nginx.frontend.prod.conf`
- `scripts/copy-docs-to-dist.js`

## Points de vigilance

### Si le changement touche seulement la doc

Le frontend n'a pas besoin d'être modifié, mais le build global doit toujours produire `dist/docs`.

### Si le changement touche Supabase

Vérifier aussi :

- migrations
- RLS
- scripts de déploiement éventuels

### Si le changement touche le backend

Vérifier :

- variables `backend/.env`
- routes exposées
- comportement Docker ou serveur cible

## Procédure minimale avant livraison

1. `npm run docs:build`
2. `npm run build`
3. si besoin, `npm --prefix backend run dev` ou vérification backend ciblée
4. vérifier que `/docs` reste accessible dans la sortie finale

# Déploiement VPS, Docker, Nginx et Firebase

## INFRASTRUCTURE

Cette page documente ce que le dépôt supporte concrètement : frontend statique, backend Node, Nginx, Docker Compose et Firebase Hosting.

## Deux stratégies visibles dans le repo

Le dépôt montre au moins deux modes de livraison :

1. frontend statique déployé via Firebase Hosting ;
2. stack VPS ou serveur avec Docker + Nginx + backend Node.

## Mode 1 — Firebase Hosting

Le fichier `firebase.json` indique :

- `dist/` comme dossier public ;
- des headers de sécurité et de cache ;
- une CSP explicite ;
- la coexistence avec les règles Firebase Database et Storage.

## Ce que cela implique

- le build frontend doit produire `dist/` ;
- la documentation copiée dans `dist/docs` peut être servie avec l'application ;
- le backend Node n'est pas servi par Firebase Hosting lui-même.

## Mode 2 — VPS / serveur Dockerisé

Le fichier `docker-compose.prod.yml` définit deux services :

- `backend`
- `frontend`

## Backend

- build à partir de `./backend`
- variables depuis `backend/.env`
- `NODE_ENV=production`
- port interne `3000`
- healthcheck via `/health`

## Frontend

- build via `Dockerfile.frontend.prod`
- variable `VITE_API_URL=/api`
- exposition du port `80`
- dépendance au backend sain

## Rôle de Nginx dans la stack VPS

Le fichier `deploy/nginx.frontend.prod.conf` montre :

- un root statique sur `/usr/share/nginx/html`
- un `try_files` pour supporter la SPA Vue
- un proxy `/api/` vers `backend:3000`
- un proxy `/health` vers le backend
- un cache long pour les assets statiques

## Lecture de la topologie VPS

Client

- Nginx (conteneur frontend)
- fichiers statiques Vue dans `/usr/share/nginx/html`
- proxy `/api/*` vers backend Node
- backend Node
- Supabase / FTP / services externes

## `Dockerfile.frontend.prod`

Le Dockerfile montre :

1. un build Node de l'application ;
2. l'injection de `VITE_API_URL` ;

3. la copie du `dist/` final dans une image Nginx ;
4. l'utilisation de la conf `deploy/nginx.frontend.prod.conf`.

## Ce qu'il faut vérifier sur un VPS

### Systeme

- Docker installé ;
- Docker Compose disponible ;
- ports réseau ouverts ;
- DNS pointant vers le serveur si domaine utilisé.

### Backend

- `backend/.env` présent ;
- variables Supabase, FTP et autres secrets définies ;
- container backend sain ;
- `/health` répond.

### Frontend

- build frontend réussi ;
- conteneur Nginx sain ;
- `/` répond ;
- `/api/ping` transite bien jusqu'au backend ;
- `/docs` fonctionne si la doc est embarquée dans `dist`.

## Procédure de déploiement VPS recommandée

1. builder localement ou sur serveur ;
2. vérifier `npm run build:all` ;
3. vérifier la présence de `dist/docs` ;
4. démarrer `docker compose -f docker-compose.prod.yml up -d --build` ;
5. tester `/health`, `/api/ping`, `/` et `/docs`.

## Différences importantes entre Firebase Hosting et VPS



| Sujet                   | Firebase Hosting          | VPS Docker    |
|-------------------------|---------------------------|---------------|
| Frontend                | oui                       | oui           |
| Backend Node            | non directement           | oui           |
| Proxy <code>/api</code> | hors Firebase Hosting pur | oui via Nginx |
| Contrôle Nginx          | non                       | oui           |
| Dépendance au serveur   | faible                    | forte         |
| Exploitation système    | faible                    | complète      |

## Réflexes en incident VPS

- tester `docker ps`
- tester `/health`
- tester `/api/ping`
- vérifier les logs du backend
- vérifier la résolution des proxys Nginx
- vérifier les secrets et la connectivité externe

## Limite de ce que montre le repo

Le dépôt montre une topologie Nginx/Docker clairement supportée.

Il ne contient pas, dans sa version active actuelle :

- de configuration Caddy active ;
- d'orchestration Kubernetes ;
- de reverse proxy externe plus détaillé que Nginx.

# Exploitation VPS et opérations

## OPÉRATIONS

Cette page vise l'exploitation concrète : build, lancement, vérification, incident et points de contrôle côté VPS.

## Hypothèse de topologie

Le repo supporte une topologie Docker simple :

- un conteneur backend Node ;
- un conteneur frontend Nginx ;
- un réseau Docker interne ;
- proxy `/api` et `/health` par Nginx.

## Fichiers à connaître

- `docker-compose.prod.yml`
- `Dockerfile.frontend.prod`
- `deploy/nginx.frontend.prod.conf`
- `backend/.env`
- `firebase.json` si mode Hosting

## Commandes de base

### Build complet

```
npm run build:all
```

### Lancement Docker prod

```
docker compose -f docker-compose.prod.yml up -d --build
```

### Vérifier les conteneurs

```
docker ps
```

## Vérifier les logs

```
docker logs pfpheds-backend  
docker logs pfpheds-frontend
```

## Contrôles post-déploiement

À tester systématiquement :

- `/`
- `/docs`
- `/health`
- `/api/ping`
- `/api/ftp/diagnostic`

## Vérifications backend

Contrôler :

- présence de `backend/.env` ;
- variables Supabase correctes ;
- éventuelles variables FTP présentes ;
- santé du backend via `/health`.

## Vérifications frontend

Contrôler :

- présence du build dans `dist/` ;
- présence de la doc dans `dist/docs` si build global ;
- bon routage SPA via Nginx ;
- disponibilité des assets statiques.

## Vérifications Nginx

La conf du repo attend :

- un root sur `/usr/share/nginx/html`
- `try_files $uri $uri/ /index.html`
- proxy `/api/` vers `backend:3000`
- proxy `/health` vers le backend

Si l'app charge mais que l'API ne répond pas, vérifier d'abord ce point.

## Symptômes courants sur VPS

### Le frontend charge mais l'app est cassée

Causes probables :

- proxy `/api` cassé ;
- backend down ;
- variables frontend mal injectées ;
- CORS si déploiement hybride.

### `/health` KO

Causes probables :

- backend non démarré ;
- env backend invalide ;
- container unhealthy ;
- port interne incorrect.

### `/docs` manquant

Causes probables :

- `npm run build:all` non exécuté ;
- `dist/docs` absent ;
- déploiement frontend fait sans copie de la documentation.

## Procédure d'incident minimale

1. vérifier `docker ps`
2. vérifier `/health`
3. vérifier `/api/ping`
4. lire les logs backend

5. lire les logs frontend

6. vérifier ensuite Supabase / FTP / service tiers

## Checklist de redéploiement

- code à jour ;
- variables présentes ;
- build app OK ;
- build docs OK ;
- conteneurs rebuildés ;
- endpoints de santé OK ;
- page `/docs` accessible.

# Runbook de maintenance et diagnostic

## EXPLOITATION

Ce runbook sert de base courte pour valider l'état de la plateforme avant d'aller plus loin dans l'analyse.

- ♦ Vérifier les endpoints
- ♦ Valider l'auth et les accès
- ♦ Contrôler la source de données
- ♦ Qualifier l'impact utilisateur

## Endpoints utiles

- `/health`
- `/api/ping`
- `/api/ftp/diagnostic`

## Contrôles rapides

1. vérifier que l'application répond encore ;
2. vérifier que les endpoints de base retournent bien un statut attendu ;
3. confirmer si le problème touche le frontend, le backend ou la donnée ;
4. vérifier si l'incident est global ou limité à un domaine métier.

# Tests et assurance qualité

## Commandes

```
npm run test:unit  
npm run test:coverage  
npm run test:e2e
```

# Scripts

## Scripts utiles

### Build docs

```
npm run docs:build  
npm run docs:serve
```

### Build complet

```
npm run build:all
```

### Tests frontend

```
npm run test:unit  
npm run test:coverage  
npm run test:e2e
```

### Tests ciblés environnement

```
node test-supabase-connection.js  
node test_postgres_connection.js  
node test_all_rpcs.js
```

## Règle

Après toute modification de `.env`, redémarrer les processus de dev.



# Outils de diagnostic

Outils et méthodes pour diagnostiquer rapidement les problèmes Firebase, Supabase et documentation.

## Diagnostics intégrés

- Supabase : ouvrir `/admin/supabase-diagnostic`
  - affiche la session, exécute des requêtes de test et remonte les erreurs
- Firebase : dans l'admin simple modules, bouton `Diagnostic Firebase`
  - vérifie les variables, teste une lecture RTDB et affiche des messages lisibles

## Console navigateur

- `firebase.js` loggue la présence des variables `VITE_FIREBASE_*`
- les guards du routeur logguent les permissions (`roleStore.perms`, `isSuper`)

## Variables d'environnement

- `.env` doit contenir :
  - Firebase : `VITE_FIREBASE_*`
  - Supabase : `VITE_SUPABASE_URL`, `VITE_SUPABASE_KEY`
- redémarrer `npm run dev` après toute modification

## Scripts utiles

- test Supabase Node : `node test-supabase-connection.js`
- build docs : `npm run docs:build`

## Problèmes fréquents

- Firebase Auth non enregistré → voir `troubleshooting/firebase-auth`
- variables d'environnement illisibles → voir `troubleshooting/firebase-env`
- URL Supabase contenant `/rest/v1` → normalisation automatique dans `src/supabase.js`

# Workflow de contribution

## Sequence recommandee

1. identifier le domaine métier
2. retrouver la route et la vue
3. confirmer la source de donnees
4. modifier store, service, vue ou backend au bon endroit
5. verifier tests et documentation

## Avant merge

- verifier le comportement reel
- lancer les tests cibles
- mettre a jour la doc si le fonctionnement change
- verifier les permissions si un ecran sensible est impacte

## Build utile

- `npm run build`
- `npm run build:all`

# Conventions projet

## Objectif

Garder des modifications lisibles dans un codebase multi-domaines.

## Conventions de base

- composants Vue en PascalCase
- logique transverse dans `composables/`
- logique métier dans `service/`
- etat partage dans `stores/`
- routes par domaine dans `src/router/routes/`

## Regle pratique

Avant d'ajouter un nouveau fichier:

1. verifier si la logique existe deja ailleurs
2. eviter de dupliquer un service Firebase et un service Supabase pour le meme besoin
3. documenter tout ajout structurant dans la doc

# ESLint et Prettier

## Commandes

```
npm run lint  
npm run format
```

## Usage

- `lint` pour les controles de code
- `format` pour normaliser le style sur `src/`

## Limite actuelle

Le formatage ne couvre pas encore toute la base de code ni la documentation. Il faut donc garder une vigilance manuelle sur les nouveaux fichiers docs.

# Guide de style de la documentation

## Objectif

Garder une documentation lisible, technique et maintenable.

## Règles simples

- un titre clair
- une introduction courte
- des sections courtes
- des listes quand elles apportent de la clarté
- des exemples réels plutôt que des phrases vagues

## Code et commandes

- utiliser des blocs de code typés
- utiliser `bash` pour les commandes
- éviter les gros extraits si un résumé suffit

## Liens

- ne lier que vers des pages existantes
- privilégier les chemins documentaires stables

## Niveau attendu

Une page doit permettre à quelqu'un de reprendre le sujet sans explication orale.

# Utiliser GitHub avec le projet

Ce guide décrit le flux GitHub du projet : cloner le dépôt, travailler sur une branche propre, ouvrir une Pull Request, puis livrer un code traçable et relisible.

## Cloner et démarrer

```
git clone https://github.com/antoinequarroz/pfpheds.git
cd pfpheds
npm install
npm run dev
```

Pour lancer la documentation locale :

```
npm run docs:dev
```

Voir aussi :

- `getting-started`
- `ops/development`
- `contrib/workflow`

## Stratégie de branches

Branches utilisées dans le dépôt :

- `main` : branche d'intégration principale
- `prod` : branche de déploiement de production
- `feature/<ticket>-<slug>` : nouvelle fonctionnalité
- `fix/<ticket>-<slug>` : correction ciblée
- `hotfix/<ticket>-<slug>` : correction urgente pour production

Exemple :

```
git checkout -b feature/JIRA-123-filtre-carte
```

## Conventions de commit

Règles recommandées :

- préfixer avec la clé Jira quand elle existe ;
- garder un message court, précis et orienté intention ;
- éviter les commits fourre-tout.

Exemples :

```
git commit -m "JIRA-123: ajoute le filtre par site"  
git commit -m "fix(auth): corrige la redirection après login"
```

## Pull Requests

Une PR propre doit contenir :

- un titre explicite ;
- une description du changement ;
- la méthode de test ;
- les impacts éventuels sur les données, droits, routes ou déploiements ;
- une mise à jour documentaire si le comportement change.

Cible habituelle :

- `main` pour le développement courant ;
- `prod` uniquement pour un hotfix ou une release contrôlée.

## Vérifications avant PR

Commandes minimales :

```
npm run lint  
npm run build  
npm run docs:build
```

Si la PR touche les flux métier, vérifier aussi :

- navigation selon les rôles ;
- appels Supabase/Firebase concernés ;
- régressions visuelles sur desktop et mobile ;

- documentation impactée.

## CI/CD

Le dépôt contient une pipeline GitHub Actions de déploiement. Le point d'entrée principal est :

```
.github/workflows/deploy-prod.yml
```

Le flux standard est :

1. build de l'application ;
2. éventuellement build de la documentation ;
3. publication des artefacts ;
4. déploiement de `dist/`.

Quand la documentation doit être livrée avec l'application, utiliser :

```
npm run build:all
```

## Documentation liée à GitHub

La documentation Docusaurus est structurée ici :

- contenu : `documentation/docs/`
- navigation : `documentation/sidebars.js`
- configuration : `documentation/docusaurus.config.js`
- assets statiques : `documentation/static/`

Toute modification fonctionnelle importante doit être accompagnée d'une mise à jour documentaire.

## Jira et traçabilité

Le projet s'appuie sur une traçabilité simple entre code et pilotage :

- la clé Jira apparaît dans le nom de branche ;
- la même clé peut apparaître dans les commits ;



- la PR reprend cette clé dans son titre si applicable.

Exemple :

- branche : `feature/JIRA-245-export-stage`
- commit : `JIRA-245: ajoute l'export CSV`
- PR : `JIRA-245: export des données de stage`

## Bonnes pratiques

- faire des PR petites et ciblées ;
- éviter les mélanges refactor + feature + fix dans la même PR ;
- documenter les décisions non évidentes ;
- ajouter des captures si l'UI change ;
- supprimer le code mort dans le même périmètre quand c'est sûr ;
- garder l'historique lisible pour qu'un autre développeur puisse reprendre sans contexte oral.

# Roadmap de la documentation

## Objectif

Finir une documentation technique exploitable sans transfert oral.

## Priorite 1

- decire plus finement les domaines formation pratique, planning et social
- cartographier les stores et services critiques
- documenter le backend Express route par route
- lister les variables d'environnement reelles par surface

## Priorite 2

- documenter les workflows annuels et secretariat
- ajouter des schémas Mermaid par flux métier
- relier les pages aux fichiers source les plus importants

## Priorite 3

- procedures de reprise incident
- checklists de release
- matrice des permissions fonctionnelles

# Changelog documentaire

2026-07-06

## Refonte de la documentation

- remplacement de l'ancienne base hétérogène par une structure de reprise projet
- nettoyage des pages numérotées et des doublons cassant le build
- ajout d'un parcours centré sur:
  - démarrage
  - architecture
  - auth et permissions
  - données et backends
  - exploitation

## Build

- ajout des scripts racine `docs:dev`, `docs:build`, `docs:serve`, `build:all`
- validation du build Docusaurus

## Regle

Cette page ne doit pas servir de journal produit complet. Elle sert uniquement à tracer les changements majeurs de la documentation elle-même.